

ÉTABLISSEMENT D'ENSEIGNEMENT SUPÉRIEUR — YNOV

Module : Architecture Logicielle & Développement · Projet de fin de module

RAPPORT TECHNIQUE — CONCEPTION & RÉALISATION

Système de gestion des ressources matérielles d'une faculté

Architecture, conception détaillée et réalisation d'un monolithe modulaire full-stack (Bun · Hono · Better Auth · Drizzle · PostgreSQL · React 19 · TanStack).

DOCUMENT

Rapport technique de réalisation

PROJET

Faculty Material Resources
Management

AUTEUR

Mohamed DOURAI

ENCADRANT

Khalid El FAHSSI

ANNÉE UNIVERSITAIRE

2025 — 2026

DATE

Juin 2026

Version 1.0 · Complément du cahier des charges et dossier de spécifications

Fiche du document

Titre	Rapport technique — Conception et réalisation du système de gestion des ressources matérielles d'une faculté
Type de document	Rapport technique (architecture, choix technologiques, conception détaillée, sécurité, tests, déploiement)
Projet	Faculty Material Resources Management — monolithe modulaire trois tiers
Dépôt du code source	https://github.com/Mohamed-1954/material-resources-management
Pile technique	Bun, Hono, Better Auth, Drizzle ORM, PostgreSQL, React 19, TanStack Router/Query/Form, Valibot, TailwindCSS, shadcn/ui, Docker
Public visé	Jury académique, encadrant, développeurs et mainteneurs du système
Statut	Validé — système réalisé, testé et déployable
Document associé	Cahier des charges et dossier de spécifications

Historique des révisions

Version	Date	Auteur	Nature des modifications
0.1	Mai 2026	Équipe projet	Première rédaction : architecture, pile technique, modèle de données.
0.8	Mai 2026	Équipe projet	Conception détaillée, sécurité, API, frontend, séquences et tests.
1.0	Juin 2026	Équipe projet	Version finale : déploiement, retours d'expérience, bilan et annexes.

Sommaire

Résumé exécutif

Introduction

Du cahier des charges à la réalisation
Objectifs du rapport
Structure du document

Vue d'ensemble de la solution réalisée

Périmètre couvert
Démarche incrémentale (R0 à R6)
Chiffres clés

Architecture logicielle

Style : le monolithe modulaire trois tiers
Justification et alternatives écartées
Vue d'ensemble
Couches internes de l'API

Pile technologique

Vue synthétique et versions
Justification des choix

Organisation du code

Monorepo et espaces de travail
Le package partagé
Diagramme de packages
Arborescence

Modèle de données

Diagramme de classes du domaine
Tables et schéma relationnel
Énumérations et statuts
Intégrité référentielle

Conception détaillée du backend

Découpage en couches
Machines à états
Algorithme du moins-disant
Transactions, audit et notifications
Validation et gestion des erreurs

Sécurité et contrôle d'accès

Authentification (Better Auth)
Matrice de permissions (RBAC)
Sessions, cookies et CORS
Limitation de débit et secrets

Traçabilité (audit)

Interface de programmation (API REST)

Conventions et enveloppe de réponse

Codes d'erreur

Catalogue des points d'accès

Diagramme de composants

Architecture frontend

Pile et principes

Routage par fichiers et garde d'accès

État serveur et état de session

Navigation par rôle et tableaux de bord

Système de composants

Diagrammes de séquence applicatifs

Tests et démarche qualité

Stratégie de test en couches

Tests des services purs

Outillage qualité et intégration continue

Déploiement et exploitation

Conteneurisation

Variables d'environnement

Mise en route

Diagramme de déploiement

Difficultés rencontrées et solutions

Bilan, limites et perspectives

Conclusion

Annexes

1. Résumé exécutif

Ce rapport technique décrit la **conception** et la **réalisation** du système de gestion des ressources matérielles d'une faculté, dont les besoins ont été formalisés dans le cahier des charges associé. Il démontre que les exigences spécifiées ont été traduites en une application full-stack fonctionnelle, sécurisée, testée et déployable.

La solution adopte une architecture en **monolithe modulaire trois tiers** : une interface web (application monopage React), une API HTTP organisée en modules métier, et une base de données relationnelle PostgreSQL. Ce style architectural a été retenu délibérément : il offre une séparation claire des responsabilités et une bonne maintenabilité, sans imposer la complexité opérationnelle d'une architecture distribuée — un compromis adapté au périmètre du projet et à un contexte académique.

Le backend repose sur l'écosystème **TypeScript/Bun** : le framework **Hono** pour le routage HTTP, **Better Auth** pour l'authentification et les sessions, **Drizzle ORM** pour l'accès typé à PostgreSQL, et **Valibot** pour la validation. Il est découpé en douze modules métier (utilisateurs, départements, besoins, appels d'offres, fournisseurs, offres, inventaire, affectations, maintenance, notifications, audit, et inscription fournisseur). La logique métier la plus sensible — les machines à états et l'algorithme de sélection du moins-disant — est isolée dans des fonctions pures, intégralement couvertes par des tests unitaires.

Le frontend est une application **React 19** bâtie sur la suite **TanStack** (Router à routage par fichiers, Query pour l'état serveur, Form pour les formulaires) et un système de composants **shadcn/ui** sur **Base UI**, stylé avec **TailwindCSS**. Chaque rôle dispose d'un tableau de bord dédié, et la navigation comme les gardes de route sont pilotées par la **même matrice de permissions** que le serveur, garantissant l'absence de divergence entre les deux couches.

La sécurité est traitée comme une préoccupation de premier ordre : authentification obligatoire, contrôle d'accès basé sur les rôles vérifié côté serveur, rôles privilégiés non auto-attribuables, sessions par cookie, limitation de débit sur l'inscription publique, absence de secrets dans le code, et journalisation d'audit des actions sensibles. L'ensemble est conteneurisé (Docker Compose : PostgreSQL, API, web) et accompagné d'une chaîne d'intégration continue, de tests automatisés et d'un jeu de données de démonstration. Ce rapport détaille chacun de ces aspects, illustrés par des diagrammes de composants, de déploiement, de classes, de séquence et des machines à états.

2. Introduction

2.1 Du cahier des charges à la réalisation

Le cahier des charges associé à ce document a établi *ce que* le système doit faire : six acteurs, six processus métier, vingt-trois exigences fonctionnelles, dix exigences non fonctionnelles et douze règles de gestion. Le présent rapport décrit *comment* ces exigences ont été satisfaites par une réalisation logicielle concrète. Il constitue donc le second volet d'une chaîne d'ingénierie continue : l'analyse a produit des spécifications, et ces spécifications ont été traduites en une architecture, une conception détaillée et un code fonctionnel.

L'enjeu n'est pas seulement de « faire fonctionner » l'application, mais de démontrer que les **choix techniques** sont justifiés, que les **règles de gestion** sont effectivement implémentées et vérifiées, et que les **exigences non fonctionnelles** (sécurité, traçabilité, intégrité, maintenabilité) sont prises en charge par construction et non par accident. Chaque section de ce rapport établit ce lien entre une décision technique et l'exigence qu'elle sert.

2.2 Objectifs du rapport

- **Justifier l'architecture** retenue et la situer par rapport aux alternatives possibles.
- **Documenter la pile technologique** et la raison de chaque choix.
- **Décrire la conception détaillée** du backend et du frontend, en particulier les mécanismes porteurs des règles métier (machines à états, sélection du moins-disant, contrôle d'accès).
- **Exposer la stratégie de sécurité** et de qualité (tests, intégration continue).
- **Expliquer le déploiement** et restituer un retour d'expérience honnête sur les difficultés rencontrées.

2.3 Structure du document

Après cette introduction, le rapport présente une vue d'ensemble de la solution réalisée (chapitre 3), puis détaille l'architecture (chapitre 4) et la pile technologique (chapitre 5). L'organisation du code est exposée au chapitre 6, le modèle de données au chapitre 7. La conception détaillée du backend (chapitre 8) et la sécurité (chapitre 9) constituent le cœur technique, complétés par l'API REST (chapitre 10) et l'architecture frontend (chapitre 11). Les diagrammes de séquence (chapitre 12) illustrent les workflows critiques. Le rapport se termine par les tests et la qualité (chapitre 13), le déploiement (chapitre 14), les difficultés rencontrées (chapitre 15), le bilan et les perspectives (chapitre 16), une conclusion (chapitre 17) et des annexes (chapitre 18).

3. Vue d'ensemble de la solution réalisée

3.1 Périmètre couvert

La solution réalisée couvre l'intégralité du périmètre fonctionnel spécifié. Les six processus métier sont opérationnels de bout en bout : un enseignant peut soumettre un besoin, un chef de département le valider, un responsable créer et publier un appel d'offre, un fournisseur soumettre une offre, le responsable sélectionner le moins-disant valide, enregistrer la livraison avec génération automatique de codes d'inventaire, affecter les ressources, et — en cas de panne — orchestrer l'intervention, le constat technique et l'action de garantie. Les notifications internes et la journalisation d'audit accompagnent l'ensemble de ces actions.

3.2 Démarche incrémentale (R0 à R6)

Le système a été construit par versions successives, chacune livrant un incrément cohérent et testé. Cette approche a permis de stabiliser les fondations transversales (authentification, contrôle d'accès, package partagé) avant d'aborder les fonctionnalités métier les plus riches.

Tableau 3.1 — Versions livrées et contenu technique.

Version	Contenu technique
R0 Fondation	Espace de travail Bun, package <code>@frms/shared</code> , configurations de base, Docker Compose, intégration continue.
R1 Auth & RBAC	Better Auth (email/mot de passe, GitHub, Google), matrice de permissions, modules utilisateurs et départements, interface adaptée au rôle.
R2 Besoins & appels	Besoins enseignants, revue/finalisation départementale, appels d'offres, publication/clôture, inclusion des besoins.
R3 Fournisseurs & offres	Inscription fournisseur, soumission d'offre, élimination, sélection du moins-disant, liste noire, acceptation/rejet, notifications.
R4 Inventaire & affectations	Enregistrement de livraison avec codes d'inventaire, spécifications ordinateur/imprimante, affectation/désaffectation avec historique.
R5 Maintenance & garantie	Signalements (contrôle d'affectation), interventions, constats techniques, réparation/remplacement sous garantie (règle de garantie active).
R6 Qualité & déploiement	Tests API et web, intégration continue, pile Docker complète, documentation académique.

3.3 Chiffres clés

Tableau 3.2 — Quelques indicateurs de la réalisation.

Indicateur	Valeur	Commentaire
Modules métier backend	12	auth, users, departments, needs, tenders, suppliers, offers, inventory, assignments, maintenance, notifications, audit
Tables relationnelles	22	4 tables d'authentification + 18 tables de domaine
Énumérations de statut	11	rôles, types, statuts (besoin, appel, offre, ressource, panne...)
Permissions distinctes	34	matrice RBAC unique partagée serveur/client
Machines à états	4	besoin, appel d'offre, offre, panne
Rôles applicatifs	6	ADMIN, RESOURCE_MANAGER, DEPARTMENT_HEAD, TEACHER, SUPPLIER, MAINTENANCE_TECHNICIAN
Fichiers de tests	15	6 côté API (services purs, RBAC, schémas) + 9 côté web

4. Architecture logicielle

4.1 Style : le monolithe modulaire trois tiers

L'application est structurée selon un **monolithe modulaire** à trois tiers. Concrètement, elle produit deux artefacts déployables — une API HTTP et une application monopage statique — accompagnés d'un package TypeScript partagé. L'API est *internement* divisée en modules métier autonomes, mais elle s'exécute en un seul processus. Ce choix est délibéré : le périmètre du projet ne justifie pas la complexité de services distribués, tandis que la séparation modulaire procure l'essentiel des bénéfices (cohésion, faible couplage, testabilité) pour une fraction du coût opérationnel.

4.2 Justification et alternatives écartées

Trois grandes options architecturales étaient envisageables :

Tableau 4.3 — Options architecturales et décision.

Option	Avantages	Inconvénients dans ce contexte	Décision
Monolithe « classique » (non modularisé)	Simplicité immédiate	Couplage fort, code « plat de spaghetti », maintenabilité dégradée à terme	Écartée
Microservices	Découplage maximal, scalabilité indépendante	Complexité opérationnelle (réseau, déploiement, observabilité, transactions distribuées) disproportionnée pour le périmètre	Écartée
Monolithe modulaire	Séparation claire par domaine, faible couplage, déploiement simple, transactions locales	Reste un seul artefact (acceptable ici)	Retenue

Le monolithe modulaire satisfait directement l'exigence de **maintenabilité** (NFR-o8) : le système est « structuré en modules simples », chaque module suivant un même squelette (routes, service, accès aux données). Il facilite aussi la **testabilité** en isolant la logique métier dans des services purs, et l'**intégrité** (NFR-o4) en conservant des transactions locales à une base unique.

4.3 Vue d'ensemble

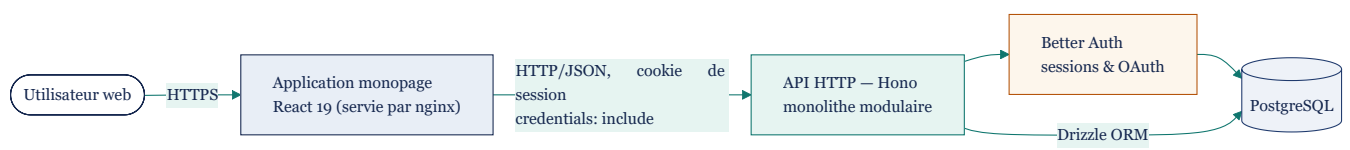


Figure 4.1 — Vue d'ensemble : trois tiers (présentation, application, données).

Le navigateur communique avec l'application monopage par HTTPS. Celle-ci dialogue avec l'API par requêtes HTTP/JSON, en transmettant systématiquement le cookie de session (option `credentials: include`, indispensable dans la configuration multi-origine du développement). L'API délègue l'authentification à Better Auth, qui persiste sessions et comptes dans PostgreSQL, et accède aux données métier via Drizzle ORM.

4.4 Couches internes de l'API

À l'intérieur du monolithe, l'API est organisée en couches transverses et en modules métier. Chaque module suit le même patron : un fichier de *routes* (exposition HTTP), un fichier de *service* (logique métier pure, fortement testée) et l'accès aux données via le schéma Drizzle partagé.

Tableau 4.4 — Couches internes de l'API et leur rôle.

Couche	Rôle
<code>auth/</code>	Instance Better Auth (adaptateur Drizzle, fournisseurs OAuth, configuration des sessions).
<code>config/</code>	Chargeur d'environnement typé.
<code>db/</code>	Schéma Drizzle, client PostgreSQL, migrations, jeu de données initial (<i>seed</i>).
<code>middleware/</code>	Hydratation de session, <code>requireAuth</code> , <code>requireRole</code> , <code>requirePermission</code> , gestionnaire d'erreurs central, limitation de débit.
<code>modules/<domaine>/*.routes.ts</code>	Points d'accès HTTP du domaine.
<code>modules/<domaine>/*.service.ts</code>	Logique métier pure : transitions d'état, algorithmes de sélection (testés unitairement).
<code>shared/</code>	Fonctions transverses : <code>errors</code> , <code>validate</code> (enveloppe Valibot), <code>audit</code> , <code>notify</code> .

PATRONS TRANSVERSES

Quatre patrons structurent le code : (1) la **matrice de permissions comme donnée** — une source unique de vérité consommée par le serveur et le client, sans branchements `if (role === ...)` dispersés ; (2) la **validation partagée** — les mêmes schémas Valibot côté serveur (obligatoire) et côté formulaire (confort) ; (3) les **machines à états** — chaque service expose un validateur de transition unique ; (4) **l'audit et les notifications** en effet de bord — canalisés par des fonctions dédiées qui journalisent sans jamais faire échouer l'opération métier.

5. Pile technologique

L'ensemble du projet repose sur un écosystème **TypeScript** unifié, du backend au frontend en passant par le package partagé. Ce choix d'un langage unique réduit la charge cognitive, permet de partager des types et des schémas de validation entre les couches, et garantit la cohérence des contrats de données. Le gestionnaire de paquets et le moteur d'exécution retenus sont **Bun**, pour sa rapidité et sa prise en charge native de TypeScript.

5.1 Vue synthétique et versions

Tableau 5.5 — Principales dépendances du backend (API).

Technologie	Version	Rôle
Bun	≥ 1.1	Moteur d'exécution, gestionnaire de paquets, lanceur de tests.
Hono	4.12.x	Framework HTTP léger (routage, middlewares, en-têtes sécurisés).
Better Auth	1.6.x	Authentification email/mot de passe et OAuth, sessions, adaptateur Drizzle.
Drizzle ORM	0.45.x	ORM typé, requêtes SQL sûres, migrations (drizzle-kit 0.31.x).
postgres (postgres-js)	3.4.x	Pilote PostgreSQL.
Valibot	1.3.x	Validation de schémas à l'exécution, légère et typée.
nanoid	5.1.x	Génération d'identifiants courts et uniques.
PostgreSQL	18 (alpine)	Système de gestion de base de données relationnelle.

Tableau 5.6 — Principales dépendances du frontend (web).

Technologie	Version	Rôle
React	19.x	Bibliothèque d'interface, composants et hooks.
TanStack Router	1.16x	Routage par fichiers, typé, avec contexte et gardes <code>beforeLoad</code> .
TanStack Query	5.x	Gestion de l'état serveur (cache, invalidation, rafraîchissement).
TanStack Form	0.4x	Gestion des formulaires et de leur validation.
Valibot	1.3.x	Mêmes schémas de validation que le serveur (via le package partagé).

Technologie	Version	Rôle
TailwindCSS	4.x	Stylage utilitaire ; thème clair/sombre.
shadcn/ui + Base UI	—	Système de composants accessibles (tableaux, dialogues, menus...).
Recharts	3.x	Graphiques des tableaux de bord (barres, secteurs).
Vite	8.x	Outil de construction et serveur de développement.
Vitest + Testing Library	2.x	Tests unitaires et de composants côté web.
oxlint / oxfmt	1.x	Analyse statique et formatage (outillage Rust, très rapide).

5.2 Justification des choix

Bun a été préféré à Node.js pour sa rapidité d'installation et d'exécution, son support TypeScript sans transpilation préalable, et son lanceur de tests intégré (`bun:test`) qui évite une dépendance supplémentaire côté API. **Hono** est un framework HTTP minimaliste, parfaitement adapté à un monolithe modulaire : il offre un système de middlewares clair (identifiant de requête, journalisation, en-têtes sécurisés, CORS, session) et une signature de routes simple à composer. **Better Auth** apporte une solution d'authentification éprouvée (sessions par cookie, OAuth GitHub/Google, hachage des mots de passe) tout en restant configurable finement, ce qui est essentiel pour interdire l'auto-attribution des rôles privilégiés.

Drizzle ORM a été choisi pour sa philosophie « proche du SQL » et son typage de bout en bout : le schéma TypeScript est la source de vérité, les requêtes sont vérifiées à la compilation, et les migrations sont versionnées. Côté frontend, la suite **TanStack** forme un ensemble cohérent : le routage par fichiers rend la structure des écrans lisible, et la séparation entre état serveur (Query) et état de formulaire (Form) clarifie la gestion d'état. Enfin, **Valibot** partagé entre les deux couches incarne le principe « un schéma, des règles identiques » : la validation côté client améliore l'expérience utilisateur, tandis que la validation côté serveur, elle, fait foi.

6. Organisation du code

6.1 Monorepo et espaces de travail

Le code est organisé en **monorepo** géré par les *workspaces* de Bun. Trois espaces cohabitent : l'API (`material-resources-management-api`), le client web (`material-resources-management-client`) et un package partagé (`packages/shared`). Cette organisation autorise un partage de code sûr (types, schémas, constantes, matrice de permissions) sans publication de paquet, et une exécution coordonnée via des scripts à la racine (`bun run dev`, `bun run test`, `bun run typecheck`, `bun run build`).

6.2 Le package partagé

Le package `@frms/shared` est la pierre angulaire de la cohérence entre serveur et client. Il exporte :

- les **constantes** et énumérations du domaine (rôles, types de ressource, statuts de besoin/appel/offre/ressource/panne, fréquences, événements de notification, codes d'erreur) ;
- la **matrice de permissions** (`PERMISSIONS`, `ROLE_PERMISSIONS`, `hasPermission`) — source unique consommée des deux côtés ;
- les **schémas Valibot** de validation des entrées ;
- les **types DTO** partagés des réponses d'API.

Cette mutualisation garantit qu'une règle d'autorisation ou un format de données ne peut pas « diverger » entre les deux couches : modifier la matrice de permissions, c'est la modifier partout à la fois.

6.3 Diagramme de packages

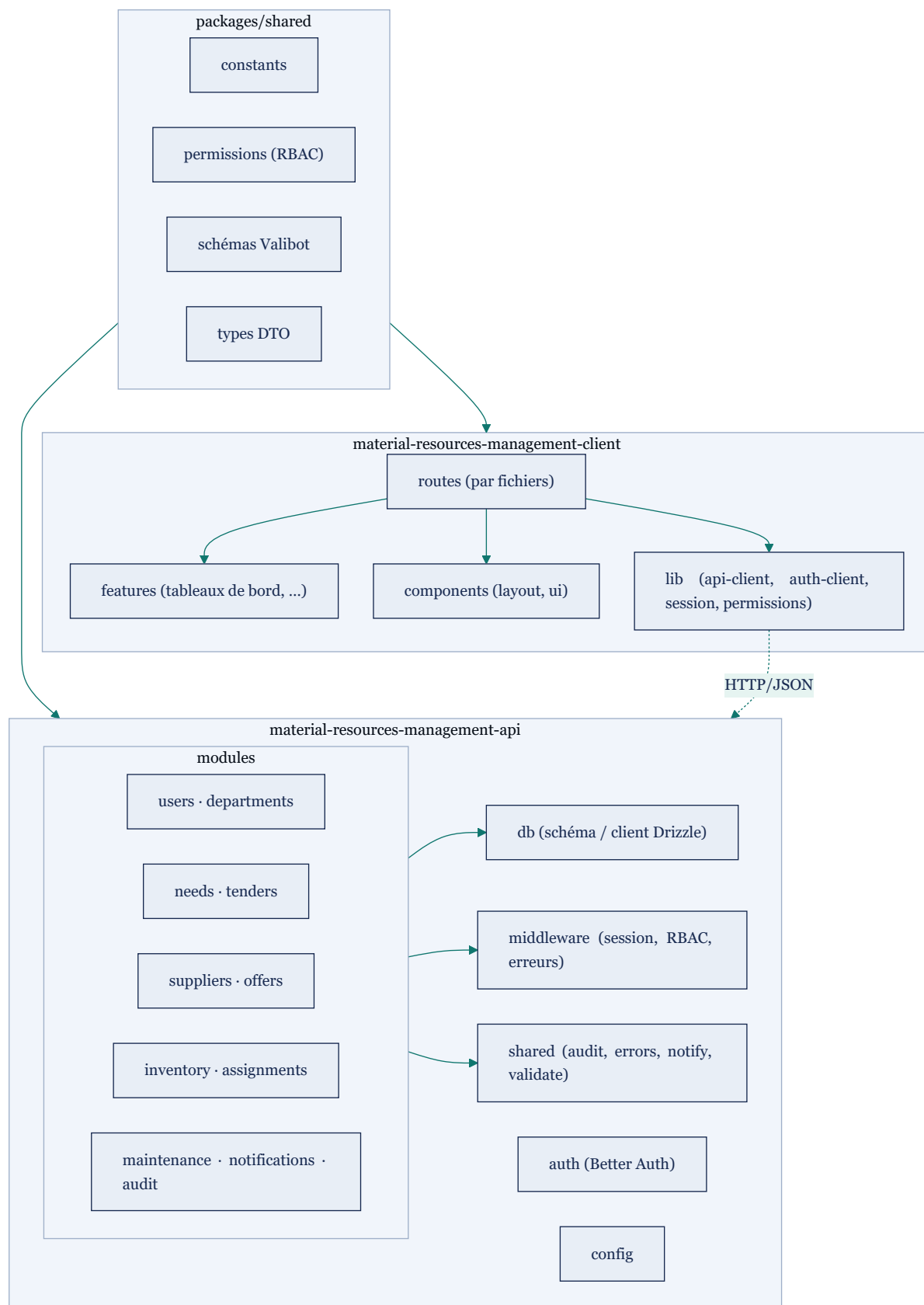


Figure 6.2 — Diagramme de packages : package partagé, API modulaire et client web.

6.4 Arborescence

La structure des répertoires reflète le découpage modulaire :

```
.
├── material-resources-management-api/      # API Hono (monolithe modulaire)
│   └── src/
│       ├── auth/          # configuration Better Auth
│       ├── config/        # chargeur d'environnement typé
│       ├── db/            # schéma Drizzle, client, migrate, seed
│       ├── middleware/    # session, RBAC, gestionnaire d'erreurs, rate-limit
│       ├── modules/       # 12 modules métier (routes + service)
│       │   ├── auth/ users/ departments/ needs/ tenders/
│       │   ├── suppliers/ offers/ inventory/ assignments/
│       │   └── maintenance/ notifications/ audit/
│       └── shared/        # audit, errors, notify, validate
├── material-resources-management-client/  # React + Vite + TanStack
│   └── src/
│       ├── components/    # layout (AppShell, sidebar), primitives ui
│       ├── features/      # tableaux de bord et écrans par domaine
│       ├── lib/           # api-client, auth-client, session, permissions
│       └── routes/        # routage par fichiers (TanStack Router)
├── packages/shared/       # schémas, types, constantes, matrice RBAC
├── infra/docker-compose.yml # postgres + api + web
├── docs/                 # documentation académique (ce rapport)
└── .github/workflows/ci.yml # intégration continue
```

7. Modèle de données

Le modèle de données traduit le langage métier en un schéma relationnel. Il se présente d'abord sous forme d'un diagramme de classes du domaine (vue conceptuelle), puis sous forme de tables PostgreSQL (vue physique) avec leurs énumérations et leurs contraintes d'intégrité.

7.1 Diagramme de classes du domaine

Pour préserver la lisibilité, le modèle de domaine est présenté en deux vues thématiques complémentaires : la chaîne d'**acquisition** (organisation, besoins, appels d'offres, offres) et le cycle de vie en **inventaire et maintenance** (ressources, affectations, pannes, garanties). Les attributs précis de chaque entité figurent dans les tables du schéma relationnel ci-après.

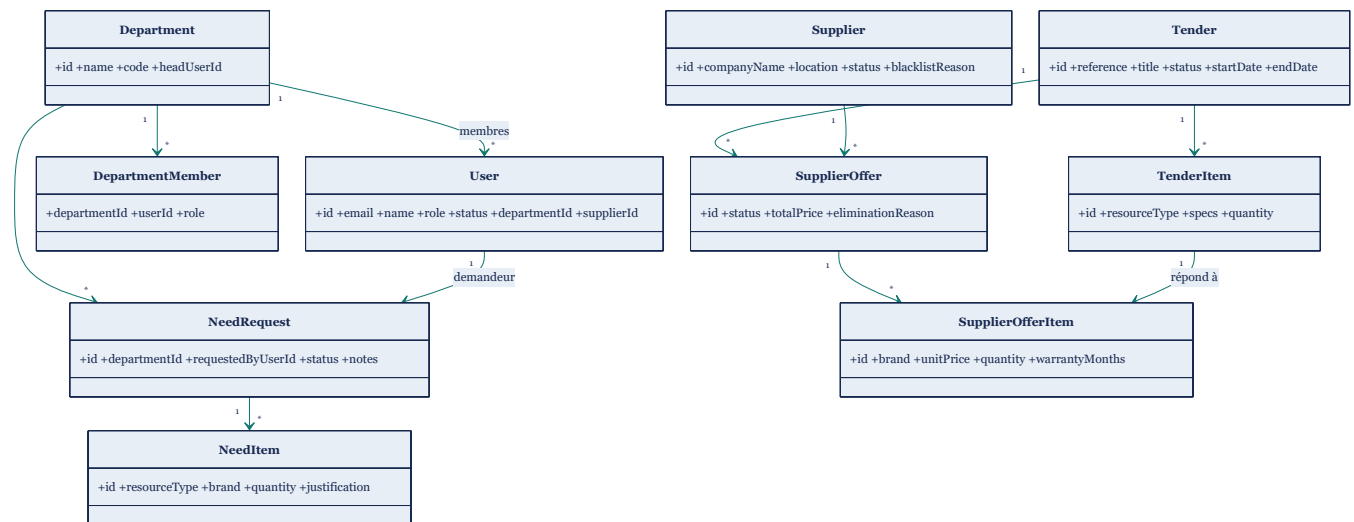


Figure 7.3 — Modèle de domaine — chaîne d'acquisition (organisation, besoins, appels d'offres, offres).

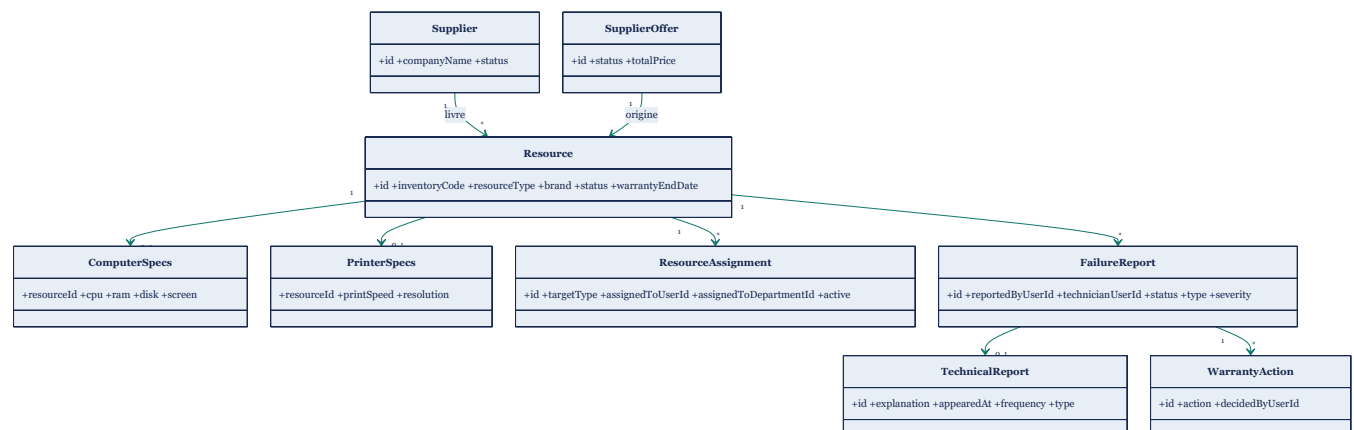


Figure 7.4 — Modèle de domaine — inventaire et maintenance (ressources, affectations, pannes, garanties).

Deux entités transverses complètent le modèle : **Notification** (id , userId , event , message , read), rattachée à chaque utilisateur, et **AuditLog** (id , userId , action , entityType , entityId ,

`oldValues` , `newValues`), qui référence n'importe quelle entité du domaine et matérialise la traçabilité.

7.2 Tables et schéma relationnel

Le schéma physique comporte vingt-deux tables : quatre tables d'authentification gérées par Better Auth (`user` , `session` , `account` , `verification`) et dix-huit tables de domaine. Les identifiants sont des chaînes (texte) ; les horodatages sont stockés avec fuseau ; les montants utilisent le type `numeric(14,2)` .

Tableau 7.7 — *Tables d'authentification et d'organisation.*

Table	Colonnes clés
<code>user</code>	id, email (unique), name, role (déf. TEACHER), status (déf. ACTIVE), departmentId ↗, supplierId ↗, emailVerified
<code>session</code>	id, userId ↗ (cascade), token (unique), expiresAt, ipAddress, userAgent
<code>account</code>	id, userId ↗ (cascade), providerId, accountId, password (haché), jetons OAuth
<code>verification</code>	id, identifier, value, expiresAt
<code>departments</code>	id, name, code (unique), headUserId ↗
<code>department_members</code>	(departmentId, userId) clé primaire composite, role (MEMBER/HEAD)
<code>suppliers</code>	id, companyName, location, address, website, managerName, status, blacklistedAt, blacklistReason, ownerUserId ↗

Tableau 7.8 — *Tables d'acquisition (besoins, appels d'offres, offres).*

Table	Colonnes clés
<code>need_requests</code>	id, departmentId ↗, requestedByUserId ↗, status, notes, submittedAt, sentToManagerAt, rejectionReason
<code>need_items</code>	id, needRequestId ↗ (cascade), resourceType, brand, cpu/ram/disk/screen, printSpeed/resolution, quantity, justification
<code>tenders</code>	id, reference (unique), title, status, startDate, endDate, awardedOfferId ↗, createdByUserId ↗
<code>tender_items</code>	id, tenderId ↗ (cascade), resourceType, brand, specs (JSON), quantity, sourceNeedItemId ↗
<code>supplier_offers</code>	id, tenderId ↗, supplierId ↗, status, totalPrice numeric(14,2), eliminationReason, rejectionReason, decidedByUserId ↗

Table	Colonnes clés
supplier_offer_items	id, offerId ↗ (cascade), tenderItemId ↗, brand, unitPrice, quantity, warrantyDurationMonths, futureDeliveryDate

Tableau 7.9 — Tables d'inventaire, d'affectation et de maintenance.

Table	Colonnes clés
resources	id, inventoryCode (unique), resourceType, brand, status, supplierId ↗, tenderId ↗, offerId ↗, deliveryDate, warrantyEndDate
computer_specs	resourceId ↗ (clé primaire, cascade), cpu, ram, disk, screen
printer_specs	resourceId ↗ (clé primaire, cascade), printSpeed, resolution
resource_assignments	id, resourceId ↗ (cascade), targetType, assignedToUserId ↗, assignedToDepartmentId ↗, assignedByUserId ↗, assignedAt, unassignedAt, active
failure_reports	id, resourceId ↗, reportedByUserId ↗, technicianUserId ↗, status, type, frequency, severity, description
technical_reports	id, failureReportId ↗ (cascade), technicianUserId ↗, explanation, appearedAt, frequency, type
warranty_actions	id, failureReportId ↗ (cascade), action (REPAIR/REPLACEMENT), reason, decidedByUserId ↗
notifications	id, userId ↗ (cascade), event, message, link, read
audit_logs	id, userId ↗, action, entityType, entityId, oldValues (jsonb), newValues (jsonb), ipAddress, userAgent

↗ : clé étrangère. « cascade » : suppression en cascade. Les colonnes `createdAt` et `updatedAt` (horodatées) sont présentes sur la plupart des tables et omises ici par souci de concision.

7.3 Énumérations et statuts

Les valeurs de statut et de type sont centralisées dans le package partagé sous forme d'énumérations, garantissant des libellés identiques côté serveur et client.

Tableau 7.10 — Énumérations du domaine et valeurs admissibles.

Énumération	Valeurs
Rôle	ADMIN, RESOURCE_MANAGER, DEPARTMENT_HEAD, TEACHER, SUPPLIER, MAINTENANCE_TECHNICIAN

Énumération	Valeurs
Type de ressource	COMPUTER, PRINTER
Statut de besoin	DRAFT, SUBMITTED, UNDER_DEPARTMENT_REVIEW, CHANGES_REQUESTED, APPROVED_BY_DEPARTMENT, SENT_TO_RESOURCE_MANAGER, INCLUDED_IN_TENDER, REJECTED
Statut d'appel d'offre	DRAFT, PUBLISHED, CLOSED, EVALUATION, AWARDED, CANCELLED
Statut d'offre	DRAFT, SUBMITTED, UNDER_REVIEW, ELIMINATED, ACCEPTED, REJECTED, WITHDRAWN
Statut de ressource	AVAILABLE, ASSIGNED, UNDER_MAINTENANCE, SENT_TO_SUPPLIER, REPLACED, RETIRED, LOST
Statut de panne	REPORTED, ASSIGNED, IN_PROGRESS, RESOLVED, SEVERE, TECHNICAL_REPORT_CREATED, SENT_TO_SUPPLIER
Type de panne	HARDWARE, SOFTWARE_SYSTEM, SOFTWARE_UTILITY
Fréquence de panne	RARE, FREQUENT, PERMANENT
Statut de fournisseur	ACTIVE, BLACKLISTED, INACTIVE
Cible d'affectation	USER, DEPARTMENT

7.4 Intégrité référentielle

L'intégrité des données (NFR-04) est garantie par plusieurs mécanismes au niveau de la base :

- **Clés étrangères** avec politiques explicites : suppression en *cascade* pour les éléments dépendants (lignes de besoin, lignes d'offre, spécifications, affectations) et *restriction* pour les références fortes (un besoin lié à un département ne peut orphceliner ce dernier).
- **Index uniques** sur les identités naturelles : `email` d'utilisateur, `code` de département, `reference` d'appel d'offre et surtout `inventoryCode` de ressource — concrétisant la règle **BR-06** d'unicité du code d'inventaire.
- **Clé primaire composite** pour l'appartenance départementale (`department_members`), empêchant les doublons d'adhésion.
- **Index de performance** sur les colonnes interrogées fréquemment (statuts, dates d'appel, `active` des affectations, `userId` des notifications), au service de l'exigence de performance (NFR-06).
- **Suppression logique** privilégiée : une ressource « retirée » passe au statut `RETIRED` plutôt que d'être effacée, et les affectations sont désactivées (`active = false`) au lieu d'être supprimées, préservant l'historique (BR-11 et la traçabilité).

8. Conception détaillée du backend

8.1 Découpage en couches

Chaque module métier respecte une séparation nette des responsabilités sur trois niveaux. Le fichier de **routes** traduit les requêtes HTTP, applique les middlewares d'authentification et de permission, valide les entrées et formate les réponses. Le fichier de **service** contient la logique métier *pure* : il ne dépend ni de HTTP, ni directement de la base, ce qui le rend testable de manière isolée. L'accès aux données s'effectue via le schéma Drizzle. Les effets de bord transverses (audit, notifications) sont délégués à des fonctions partagées. Cette discipline permet de concentrer le risque métier dans des services testés à 100 % et de garder des routes minces.

8.2 Machines à états

Le cœur du domaine repose sur quatre **machines à états**. Chaque service expose un validateur de transition unique (par exemple `assertCanTransition` pour les besoins, `assertTenderTransition` pour les appels, `assertOfferTransition` pour les offres) : c'est le *seul* endroit où une transition est acceptée ou refusée. Toute tentative de transition illégale lève une erreur métier. Ce mécanisme garantit qu'aucun objet ne peut atteindre un état incohérent, quel que soit le chemin emprunté par l'appelant.

Cycle de vie d'un besoin

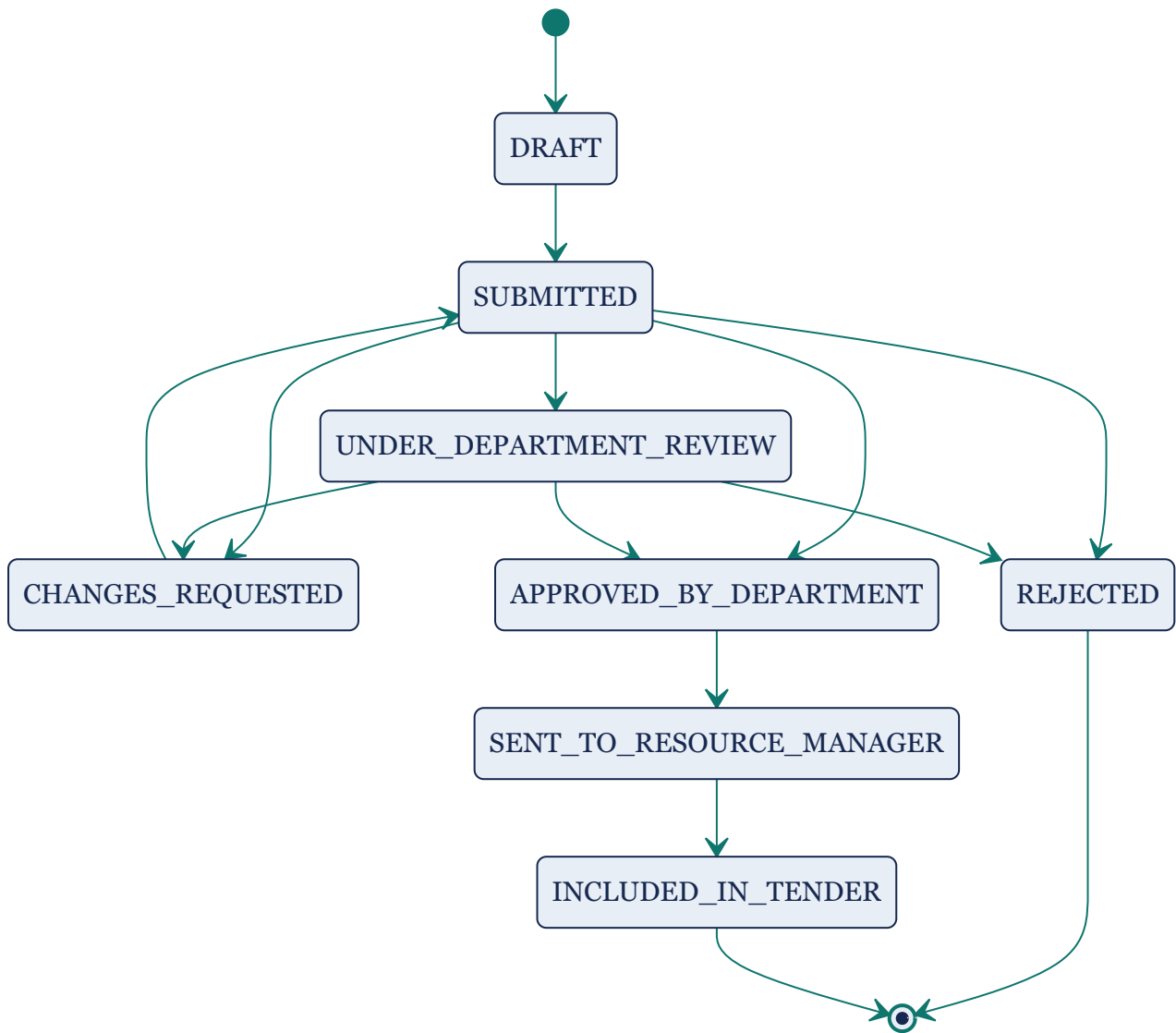


Figure 8.5 — *Machine à états du besoin.*

Cycle de vie d'un appel d'offre et d'une offre

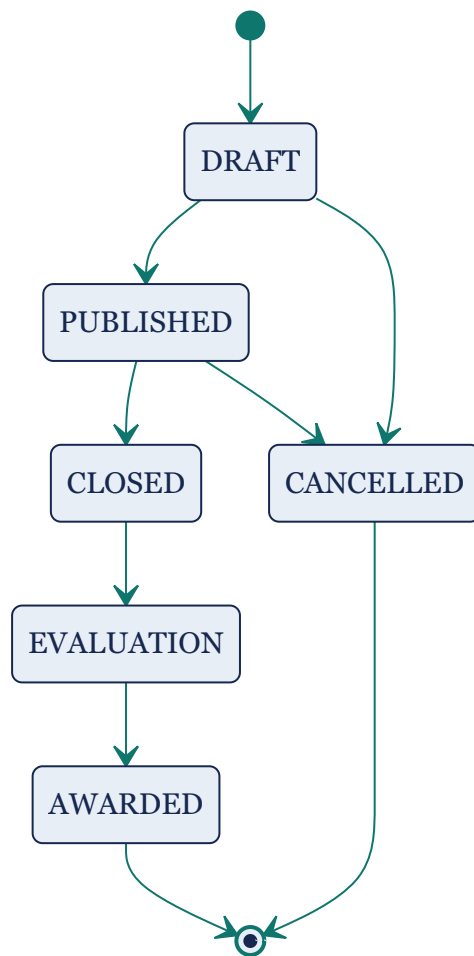


Figure 8.6 — Machine à états de l'appel d'offre.

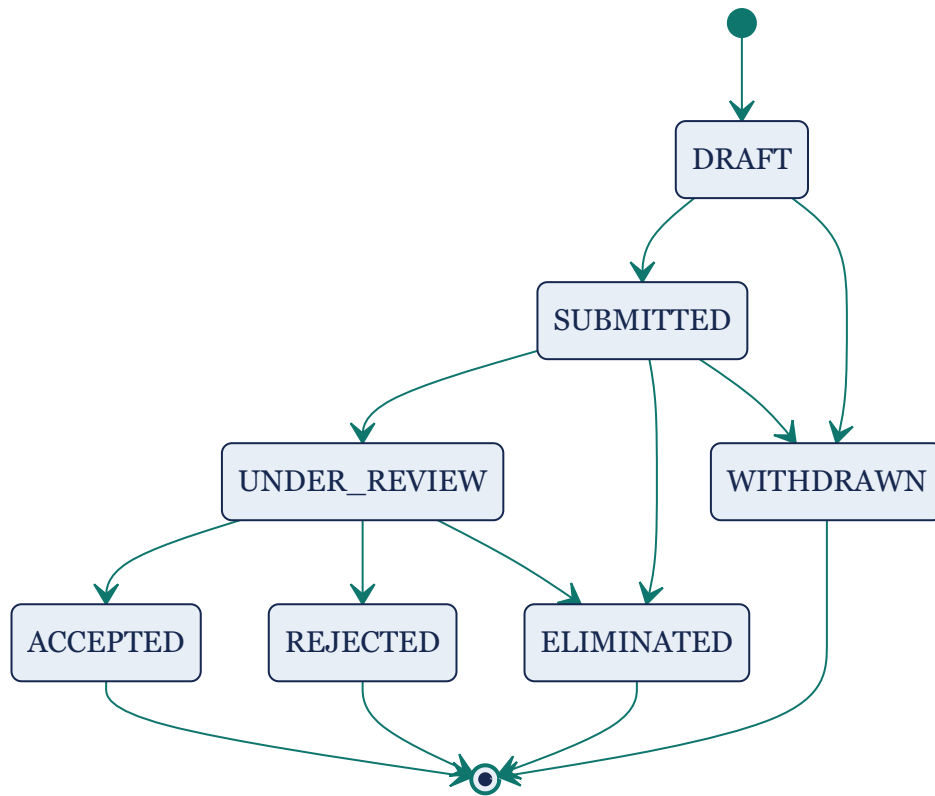


Figure 8.7 — *Machine à états de l'offre fournisseur.*

Cycle de vie d'une panne

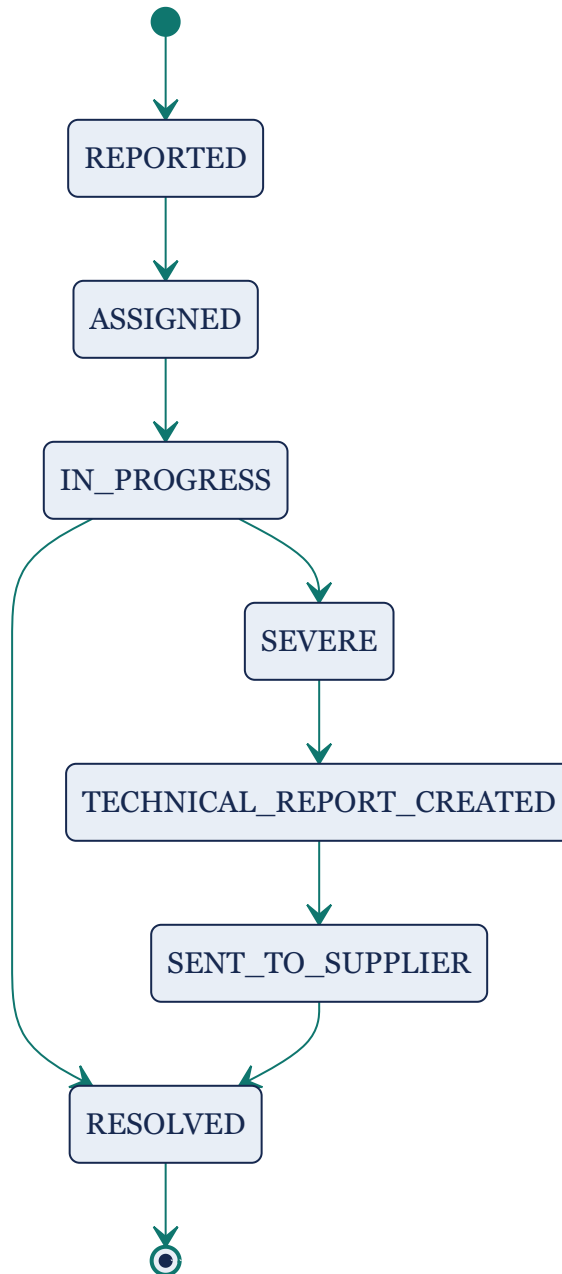


Figure 8.8 — Machine à états de la panne (du signalement à la garantie).

8.3 Algorithme du moins-disant valide

La sélection de l'offre gagnante implémente les règles **BR-05**, **BR-03** et **BR-12**. La fonction pure `selectLowestValidOffer` filtre d'abord les offres *valides* — celles dont le statut est `SUBMITTED` ou `UNDER_REVIEW` et dont le fournisseur n'est pas en liste noire — puis retient celle dont le total est le plus bas. Lors de l'acceptation, le serveur revérifie que l'offre candidate est bien la moins disante valide : si une offre valide strictement inférieure existe, l'acceptation est rejetée par une violation de règle métier.


```
function selectLowestValidOffer(offers):
  candidates = offers.filter(o =>
    (o.status == SUBMITTED || o.status == UNDER_REVIEW)
    && o.supplierStatus == ACTIVE) // exclut blacklistés, éliminés, rejetés
  if candidates.isEmpty(): return null
  return candidates.minBy(o => o.totalPrice) // moins-disant valide
```

Extrait 8.1 — Sélection de l'offre valide la moins disante (logique pure, testée unitairement).

Lorsqu'une offre est acceptée, la même transaction fait passer l'offre retenue à `ACCEPTED`, bascule les autres offres ouvertes en `REJECTED`, attribue l'appel (`AWARDED`) et notifie les fournisseurs concernés. Le total de chaque offre est calculé côté serveur par `computeOfferTotal` (somme des *prix unitaire* × *quantité*), de sorte que les valeurs saisies par le fournisseur restent purement indicatives.

8.4 Transactions, audit et notifications

Les opérations métier qui touchent plusieurs tables (acceptation d'offre, enregistrement de livraison, transitions assorties d'effets) sont exécutées dans une **transaction** de base de données. Les helpers d'audit (`recordAudit`) et de notification (`notifyMany`) acceptent un exécuteur transactionnel optionnel, de sorte que la trace et les notifications participent à la même transaction lorsque c'est pertinent. Un principe important gouverne ces effets de bord : **l'audit et la notification ne doivent jamais faire échouer une opération métier**. En dehors d'une transaction, leur échec est journalisé mais n'interrompt pas le flux ; cette robustesse évite qu'un incident périphérique ne bloque une décision d'achat ou de garantie.

8.5 Validation et gestion des erreurs

Toutes les entrées sont validées au moyen de schémas Valibot via un utilitaire `parseOrThrow` qui lève une erreur de validation structurée en cas d'échec. Les erreurs applicatives sont modélisées par une hiérarchie de fabriques (`ValidationError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`, `ConflictError`, `BusinessRuleError`, `TooManyRequestsError`) associées à des codes stables. Un gestionnaire d'erreurs central transforme toute exception en une réponse JSON normalisée, incluant un identifiant de requête pour la corrélation des journaux. Cette uniformité simplifie le traitement des erreurs côté client.

POURQUOI DES SERVICES PURS ?

En isolant les transitions d'état et l'algorithme de sélection dans des fonctions sans dépendance à HTTP ou à la base, on obtient une logique métier **déterministe et testable** sans infrastructure. C'est ce qui permet d'affirmer, preuve à l'appui, que « l'appel d'offre suit son cycle de vie » ou que « le moins-disant valide gagne » : chaque transition et chaque cas de sélection est épinglé par un test unitaire.

9. Sécurité et contrôle d'accès

La sécurité est une exigence transverse de premier rang (NFR-01, NFR-02). Elle est traitée à plusieurs niveaux : authentification, autorisation par rôle, gestion des sessions, protection des origines, limitation de débit, gestion des secrets et traçabilité.

9.1 Authentification (Better Auth)

L'authentification est confiée à **Better Auth**, configuré avec un adaptateur Drizzle sur PostgreSQL. Trois méthodes sont prises en charge : email/mot de passe (mots de passe hachés, longueur minimale imposée), et les fournisseurs OAuth GitHub et Google (activés conditionnellement selon la présence des identifiants). Les sessions ont une durée de vie de sept jours avec rafraîchissement quotidien.

Un point de conception **essentiel pour la sécurité** concerne les champs additionnels de l'utilisateur (`role`, `status`, `departmentId`, `supplierId`) : ils sont déclarés avec l'option `input: false`, ce qui signifie que Better Auth *refuse toute valeur fournie par le client*. De plus, un hook de base de données (`user.create.before`) force le rôle `TEACHER` et le statut `ACTIVE` à la création. Conséquence directe : **aucun rôle privilégié ne peut être auto-attribué** lors d'une inscription publique. Seuls les comptes fournisseurs sont créés par auto-inscription (via un point d'accès dédié) ; tous les comptes internes sont provisionnés par l'administrateur ou par le jeu de données initial.

9.2 Matrice de permissions (RBAC)

Le contrôle d'accès repose sur une **matrice de permissions traitée comme une donnée**, définie une seule fois dans le package partagé et consommée à la fois par les middlewares du serveur (`requirePermission`) et par les gardes de route du client. Il n'existe aucun branchement `if (role === ...)` dispersé dans le code métier. Trente-quatre permissions distinctes sont réparties par domaine ; l'administrateur les possède toutes.

Tableau 9.11 — Synthèse de la matrice RBAC par rôle.

Rôle	Permissions accordées (synthèse)
ADMIN	Toutes les permissions (gestion des utilisateurs et rôles, départements, audit, et l'ensemble des permissions métier).
RESOURCE_MANAGER	Voir tous les besoins et les inclure ; gérer les appels d'offres ; voir et évaluer les offres ; gérer les fournisseurs et la liste noire ; gérer et affecter les ressources ; voir toutes les pannes ; décider des garanties ; consulter l'audit.
DEPARTMENT_HEAD	Voir, examiner et finaliser les besoins de son département ; voir les ressources et les pannes de son département.

Rôle	Permissions accordées (synthèse)
TEACHER	Créer et voir ses propres besoins ; voir ses ressources ; signaler et voir ses propres pannes.
SUPPLIER	Voir les appels actifs ; créer et voir ses propres offres ; mettre à jour son profil.
MAINTENANCE_TECHNICIAN	Voir toutes les pannes ; intervenir ; rédiger des constats techniques.

Les permissions sont nommées selon une convention `domaine:action[:portée]` — par exemple `need:create:own`, `offer:evaluate`, `supplier:blacklist`, `resource:assign`, `audit:view`. La portée (`own`, `department`, `all`) encode la granularité de visibilité, ce qui satisfait l'exigence de confidentialité (NFR-10) : un fournisseur ne dispose que de `offer:view:own` et ne peut donc jamais consulter les offres concurrentes.

DÉFENSE EN PROFONDEUR

Le contrôle d'accès est appliqué **côté serveur**, où il fait foi. Le client applique les mêmes règles (masquage de boutons, gardes de route) uniquement pour le confort d'usage. Même si un utilisateur contournerait l'interface, l'API refuserait toute action non autorisée. Les deux couches partageant la *même* matrice, elles ne peuvent pas diverger.

9.3 Sessions, cookies et CORS

Les sessions sont matérialisées par des cookies signés (préfixe `frms`), marqués `httpOnly` et `sameSite: lax`. L'attribut `secure` est activé automatiquement lorsque l'URL est en HTTPS, et désactivé en développement local (HTTP) afin que le cookie de session soit bien conservé. Le partage de ressources entre origines (CORS) est configuré pour n'autoriser que les origines de confiance (`TRUSTED_ORIGINS`) et pour accepter les requêtes authentifiées (`credentials: true`). Côté client, chaque appel transmet `credentials: include` afin de propager le cookie dans la configuration multi-origine du développement.

9.4 Limitation de débit et secrets

L'inscription publique des fournisseurs est protégée par une **limitation de débit** (seau à jetons en mémoire, par adresse IP) afin de prévenir les abus ; un dépassement renvoie le code `RATE_LIMITED`. Aucun secret n'est stocké dans le code : la clé de signature des sessions (`BETTER_AUTH_SECRET`), la chaîne de connexion à la base et les identifiants OAuth proviennent exclusivement de variables d'environnement, validées au démarrage. La liste de contrôle de durcissement pour la production prévoit l'activation des cookies `secure`, l'exécution derrière un proxy TLS, la restriction des origines de confiance et la rotation des secrets.

9.5 Traçabilité (audit)

Toutes les actions sensibles — changement de rôle, mise en liste noire, élimination/acceptation/rejet d'offre, décision de garantie, affectation, transitions importantes — sont consignées dans la table `audit_logs` avec l'auteur, l'action, l'entité visée et les valeurs avant/après. Cette journalisation répond aux exigences de traçabilité (NFR-03) et d'auditabilité (NFR-05) : chaque décision importante peut être justifiée a posteriori. Le journal d'audit n'est consultable que par les rôles disposant de la permission `audit:view`.

10. Interface de programmation (API REST)

10.1 Conventions et enveloppe de réponse

Tous les points d'accès métier sont préfixés par `/api/*` ; Better Auth est monté sous `/api/auth/*`. Les réponses suivent une **enveloppe uniforme** : en cas de succès, un objet `{ data, meta? }` (où `meta` porte la pagination éventuelle) ; en cas d'erreur, un objet `{ error: { code, message, details? } }`. Cette régularité permet au client de traiter les réponses de façon générique : un utilitaire extrait `data` et lève une erreur typée porteuse du code et du message en cas d'échec.

10.2 Codes d'erreur

Tableau 10.12 — Codes d'erreur normalisés et statut HTTP associé.

Code	HTTP	Signification
VALIDATION_ERROR	400	Entrée invalide (schéma Valibot non satisfait).
UNAUTHORIZED	401	Authentification requise ou session absente.
FORBIDDEN	403	Action non autorisée pour le rôle.
NOT_FOUND	404	Entité introuvable.
CONFLICT	409	Conflit d'état (par ex. unicité violée).
BUSINESS_RULE_VIOLATION	422	Règle métier non respectée (fenêtre d'appel, moins-disant, garantie...).
RATE_LIMITED	429	Trop de requêtes (limitation de débit).
INTERNAL_SERVER_ERROR	500	Erreur interne non prévue.

10.3 Catalogue des points d'accès

Le tableau ci-dessous synthétise les principaux points d'accès par module. Les transitions d'état sont exposées comme des actions explicites (verbe métier dans le chemin) plutôt que comme de simples mises à jour, ce qui rend l'intention visible et auditable.

Tableau 10.13 — Catalogue synthétique des points d'accès de l'API.

Module	Points d'accès représentatifs
Auth	POST <code>/api/auth/sign-in/email</code> , <code>/sign-up/email</code> , GET <code>/api/auth/session</code> , POST <code>/api/auth-extras/register-supplier</code> (public, limité)

Module	Points d'accès représentatifs
Utilisateurs	GET /api/users , /users/me , POST /api/users , PATCH /api/users/:id/role , /status
Départements	GET/POST /api/departments , /:id/members , PATCH /:id/head
Besoins	GET/POST /api/needs , POST /:id/{submit approve reject request-changes send-to-resource-manager}
Appels d'offres	GET /api/tenders , /active , POST /:id/{publish close start-evaluation cancel} , /include-needs
Fournisseurs	GET /api/suppliers , /me , POST /:id/{blacklist remove-from-blacklist}
Offres	POST /api/offers/by-tender/:id , POST /:id/{submit withdraw eliminate accept reject}
Ressources	GET /api/resources , /available , POST /register-delivery , DELETE /:id (retrait logique)
Affectations	GET /api/assignments/my , /by-{user department resource}/:id , POST /by-resource/:id et /unassign
Maintenance	GET/POST /api/failures , POST /:id/{assign-technician start-intervention resolve mark-severe technical-report request-supplier-repair request-replacement}
Notifications	GET /api/notifications , /unread , PATCH /:id/read , /read-all
Audit	GET /api/audit-logs (permission audit:view) , /by-entity/:type/:id

10.4 Diagramme de composants

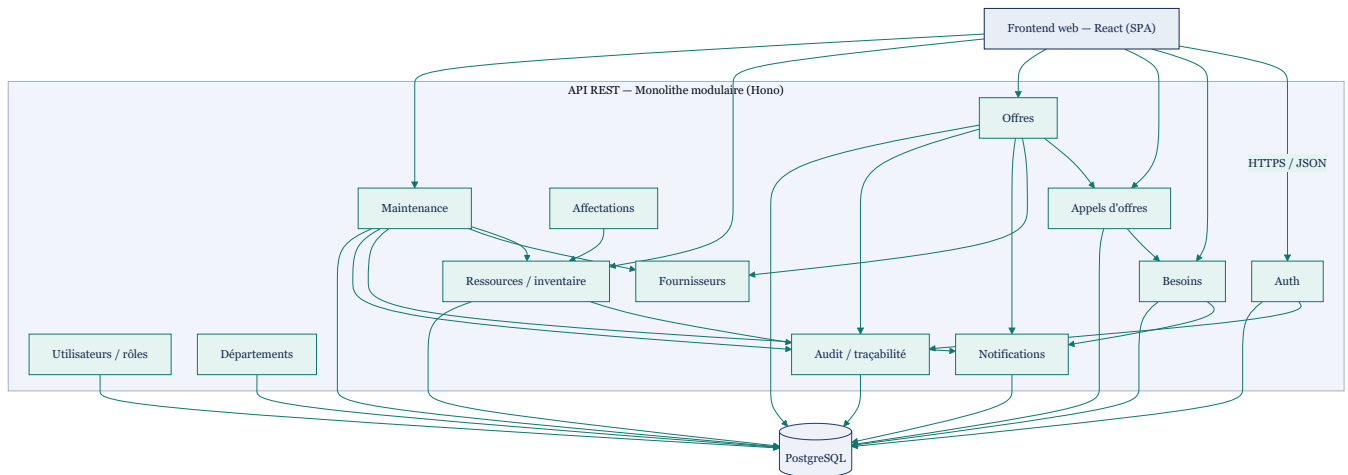


Figure 10.9 — Diagramme de composants : frontend, modules backend et base de données.

11. Architecture frontend

11.1 Pile et principes

Le frontend est une application monopage **React 19** construite avec **Vite**. Trois principes la structurent : un **roulage par fichiers** (l'arborescence des fichiers de route définit la navigation), une séparation explicite entre **état serveur** (TanStack Query) et **état de formulaire** (TanStack Form), et un **système de composants** accessible (shadcn/ui sur Base UI, stylé avec TailwindCSS). L'application est servie en production sous forme de fichiers statiques par un serveur nginx.

11.2 Routage par fichiers et garde d'accès

Le routeur (TanStack Router) génère l'arbre des routes à partir des fichiers du dossier `routes/`. La racine `__root.tsx` exécute un `beforeLoad` asynchrone qui résout la session de l'utilisateur *avant* tout rendu, et l'injecte dans le contexte du routeur. Une route de mise en page sans chemin, `_authenticated.tsx`, protège l'ensemble des écrans authentifiés : si la session est absente, elle redirige vers `/login` en mémorisant la destination initiale. Chaque écran spécifique à un rôle déclare en plus, dans son `beforeLoad`, les rôles autorisés via une fabrique de gardes `requireRole(...)` / `requirePermission(...)` qui s'appuie sur la matrice partagée. C'est une défense en profondeur : le serveur reste l'autorité, mais le client évite d'afficher des écrans inaccessibles.

```
// route-guards.ts – fabrique de garde (extrait simplifié)
export const requireRole = (...roles) => gate({ roles })
function gate(opts) {
  return ({ context, location }) => {
    if (!context.auth.isAuthenticated)
      throw redirect({ to: "/login", search: { redirect: location.href } })
    if (opts.roles && !opts.roles.includes(context.auth.user.role) && !isAdmin)
      throw redirect({ to: "/dashboard" })
  }
}
```

Extrait 11.2 — Garde de route basée sur la matrice partagée (l'ADMIN contourne les gardes de rôle).

11.3 État serveur et état de session

L'état serveur est entièrement géré par **TanStack Query** : chaque liste ou détail est une *query* identifiée par une clé hiérarchique, avec une durée de fraîcheur (`staleTime`) calibrée selon la volatilité de la donnée (par exemple plus courte pour les pannes et notifications, plus longue pour les référentiels comme les départements). La **session** elle-même est stockée dans le cache de Query sous la clé `["auth", "session"]`, ce qui évite un arbre de contexte d'authentification séparé : le routeur lit la session depuis le cache, et son rafraîchissement automatique (au retour de focus) maintient l'état à jour. Cette unification de l'état d'authentification dans le cache constitue un choix de conception à la

fois élégant et — comme l'illustre le chapitre des difficultés rencontrées — exigeant en rigueur sur la sémantique de fraîcheur.

11.4 Navigation par rôle et tableaux de bord

La barre latérale est **filtrée selon le rôle** de l'utilisateur connecté : une structure de navigation déclarative (`NAV_SECTIONS`) associe à chaque section et à chaque entrée la liste des rôles autorisés ; seules les sections comportant au moins une entrée visible sont affichées. Chaque rôle dispose en outre d'un **tableau de bord dédié**, chargé paresseusement, qui agrège les données pertinentes en graphiques (Recharts) :

Tableau 11.14 — Tableaux de bord par rôle et indicateurs présentés.

Rôle	Contenu du tableau de bord
Administrateur	Répartition des utilisateurs par rôle (secteurs), activité d'audit sur 14 jours (barres), dernières entrées d'audit.
Responsable	Pipeline des appels d'offres et résultats des offres (barres), listes des appels / besoins / fournisseurs.
Chef de département	Besoins par étape du cycle (secteurs), liste des membres du département.
Enseignant	Activité personnelle (besoins et pannes par semaine), mes besoins, mes ressources, actions rapides.
Fournisseur	Issues des offres (secteurs), taux d'acceptation, appels actifs.
Technicien	Pannes par statut (barres), constats à produire.

11.5 Système de composants

Les écrans s'appuient sur une bibliothèque de composants accessibles (shadcn/ui sur Base UI). Deux composants transverses méritent d'être cités : `DataTable` , un tableau générique gérant les états de chargement et les états vides, et `StatusBadge` , qui transforme un statut technique en pastille colorée lisible. La mise en page authentifiée (`AppShell`) fournit une barre latérale, un fil d'Ariane, une cloche de notifications et un menu utilisateur. Conformément à l'exigence d'ergonomie (NFR-09), les tableaux affichent des libellés humains (nom, email, référence, code d'inventaire) plutôt que des identifiants techniques, et les parcours sont jalonnés de messages d'erreur explicites et d'états de chargement.

COHÉRENCE CLIENT / SERVEUR

Le frontend importe le *même* package `@frms/shared` que l'API : mêmes énumérations, mêmes schémas de validation, même matrice de permissions. Un formulaire valide donc les saisies avec exactement les règles que le serveur appliquera, et la navigation reflète exactement les droits que le serveur autorisera. Cette source unique de vérité élimine une classe entière de bogues de désynchronisation.

12. Diagrammes de séquence applicatifs

Les diagrammes de séquence décrivent la dynamique des workflows critiques : l'enchaînement des messages entre l'utilisateur, l'interface, les services métier, la base de données et les services transverses (authentification, notifications). Ils explicitent les points de décision (blocs *alt*) et les répétitions (blocs *loop*), et montrent où s'appliquent les règles de gestion.

12.1 Authentification et établissement de session

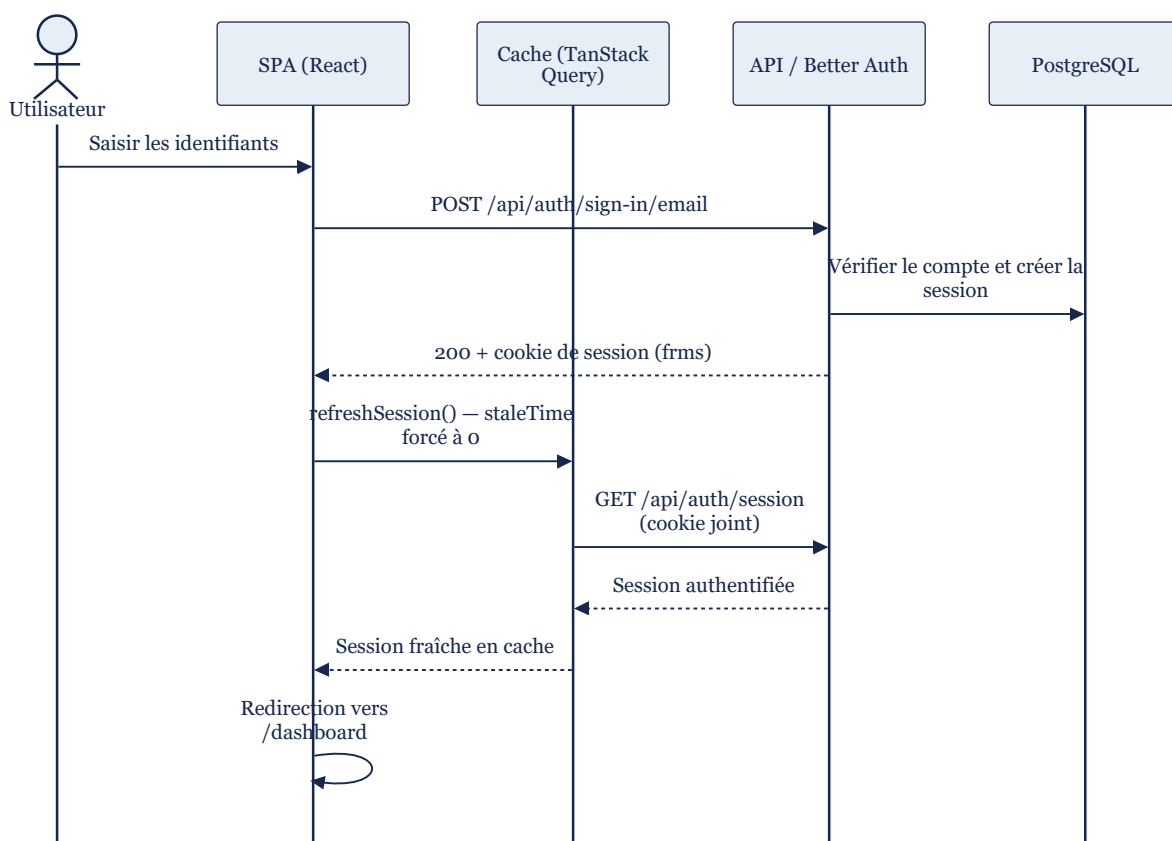


Figure 12.10 — Authentification : connexion, établissement de session et redirection.

12.2 Soumission d'un besoin

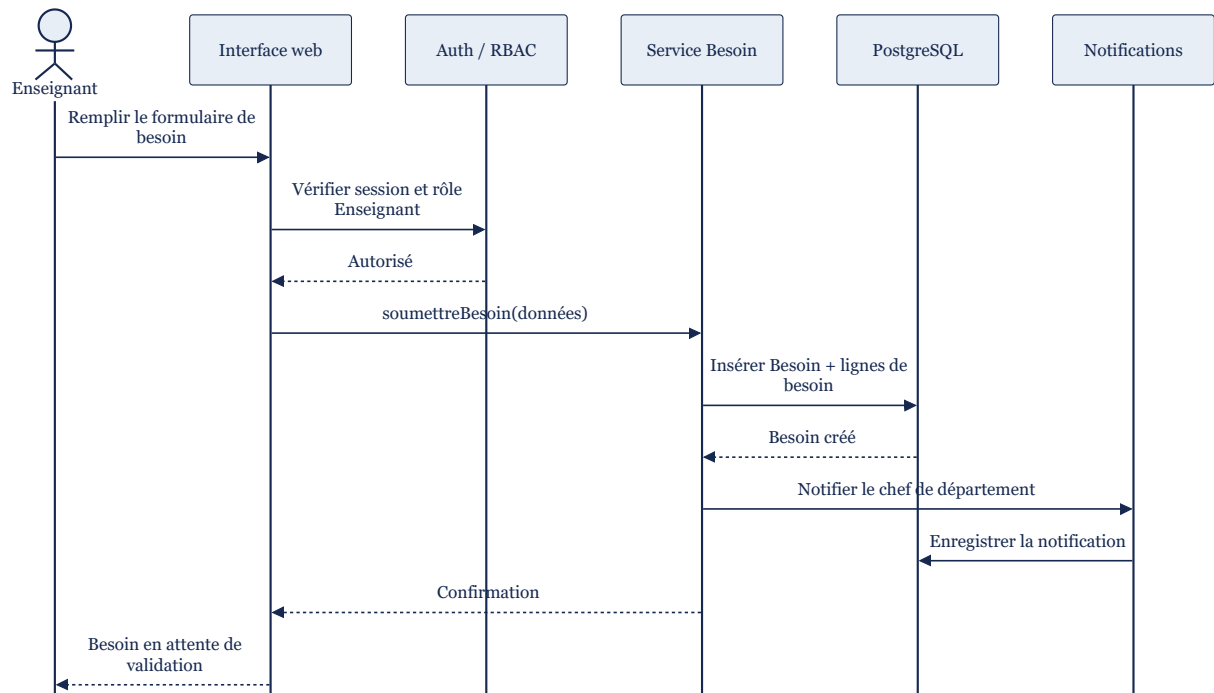


Figure 12.11 — Soumission d'un besoin par un enseignant.

12.3 Création et publication d'un appel d'offre

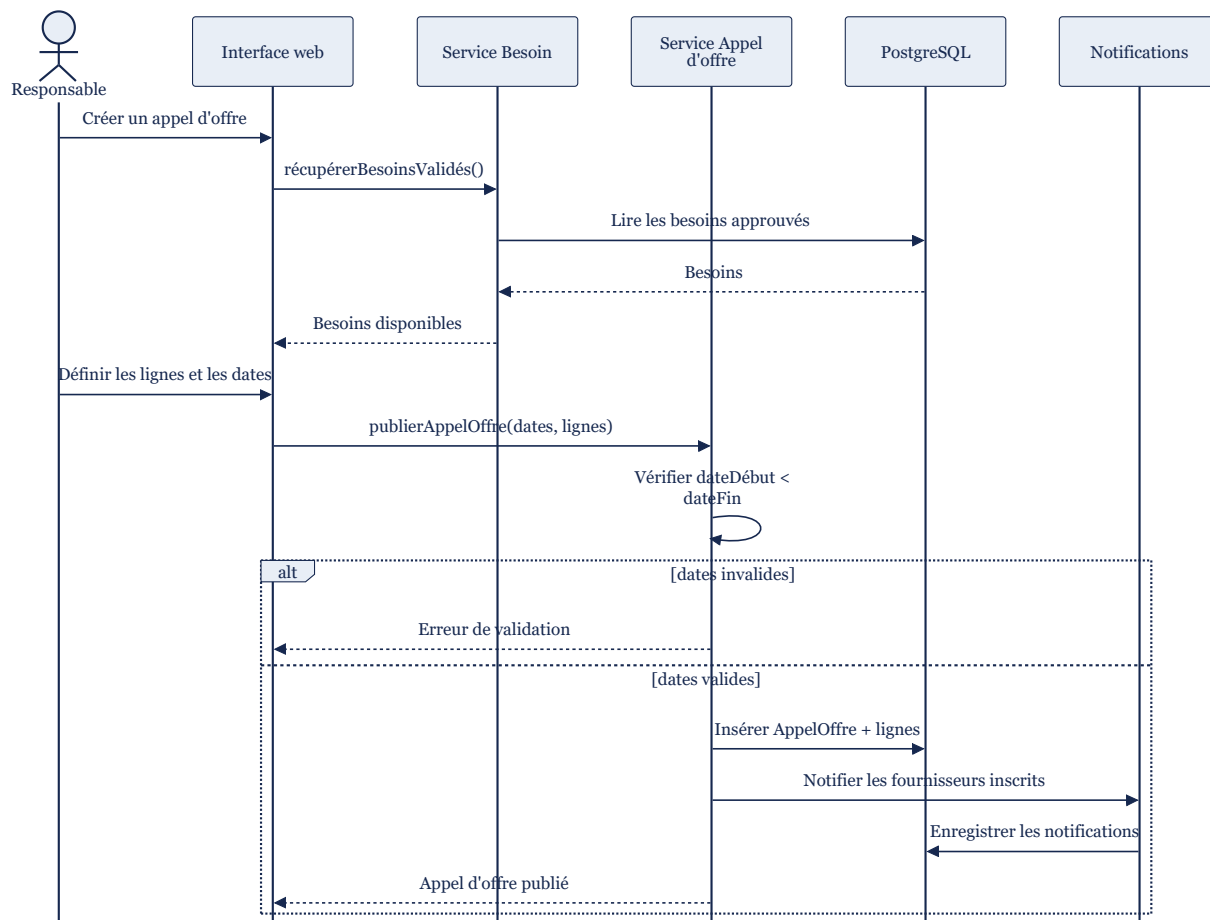


Figure 12.12 — Création d'un appel d'offre avec contrôle de la fenêtre temporelle (BR-01).

12.4 Soumission d'une offre

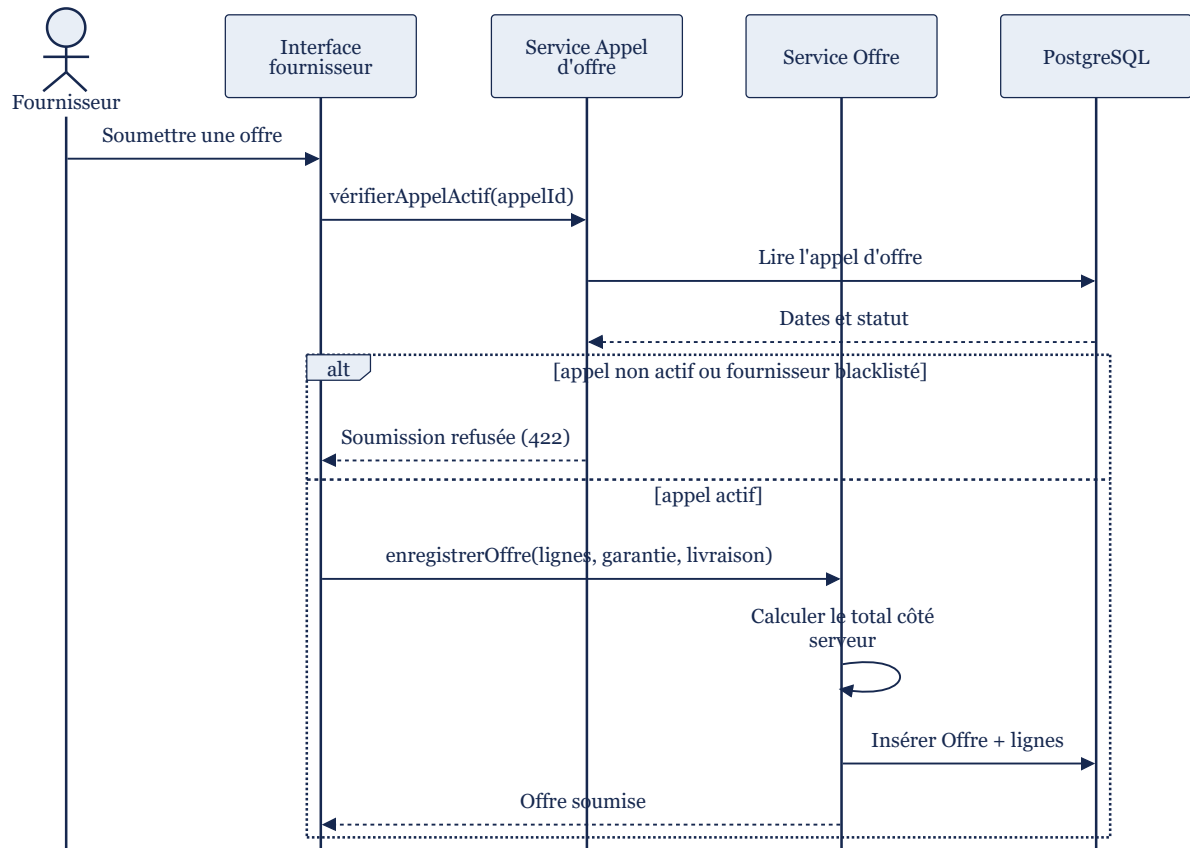


Figure 12.13 — Soumission d'une offre, conditionnée par la fenêtre active (BR-02).

12.5 Évaluation et sélection du moins-disant

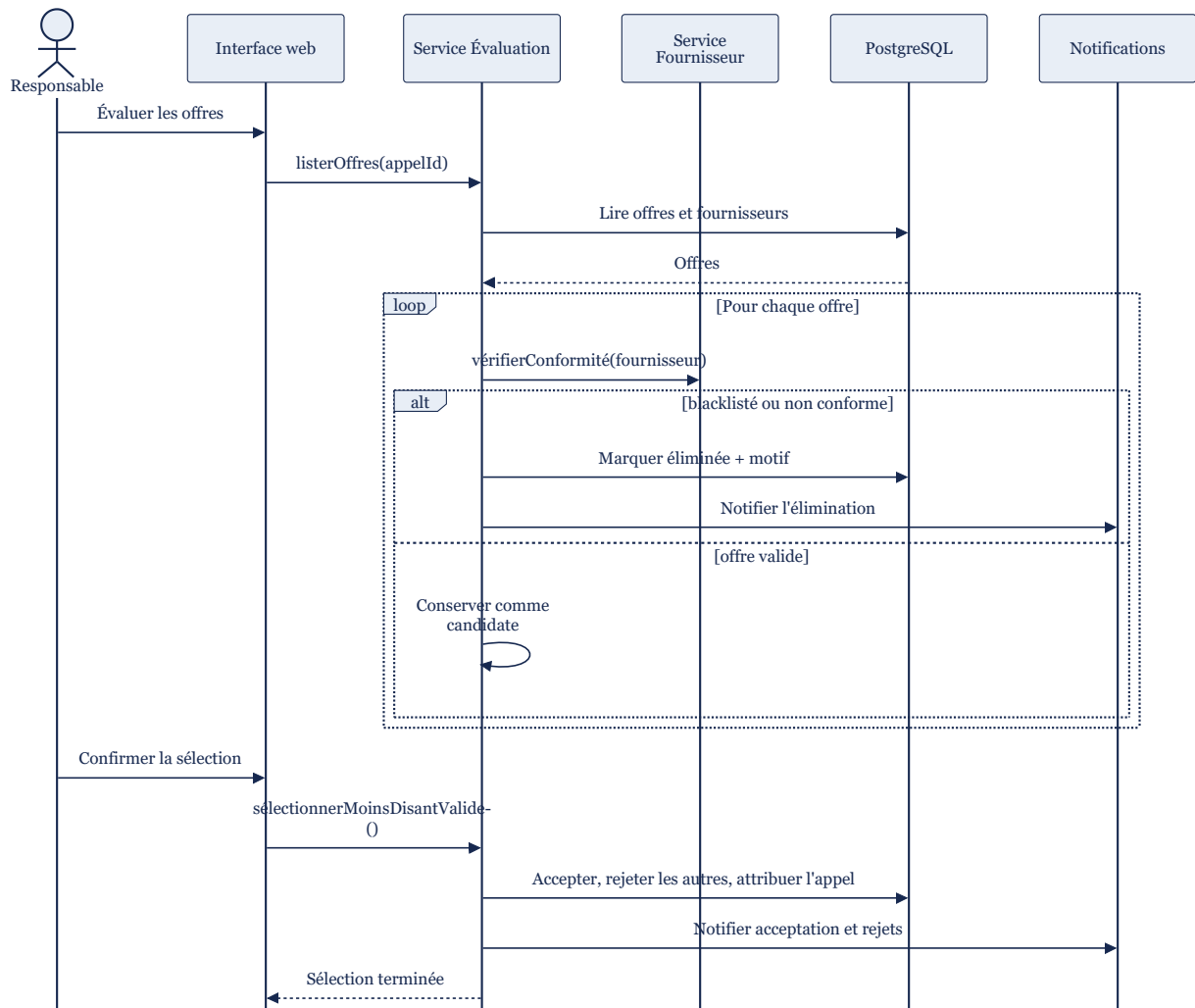


Figure 12.14 — Évaluation des offres et sélection du moins-disant valide (BR-04, BR-05, BR-12).

12.6 Livraison et affectation

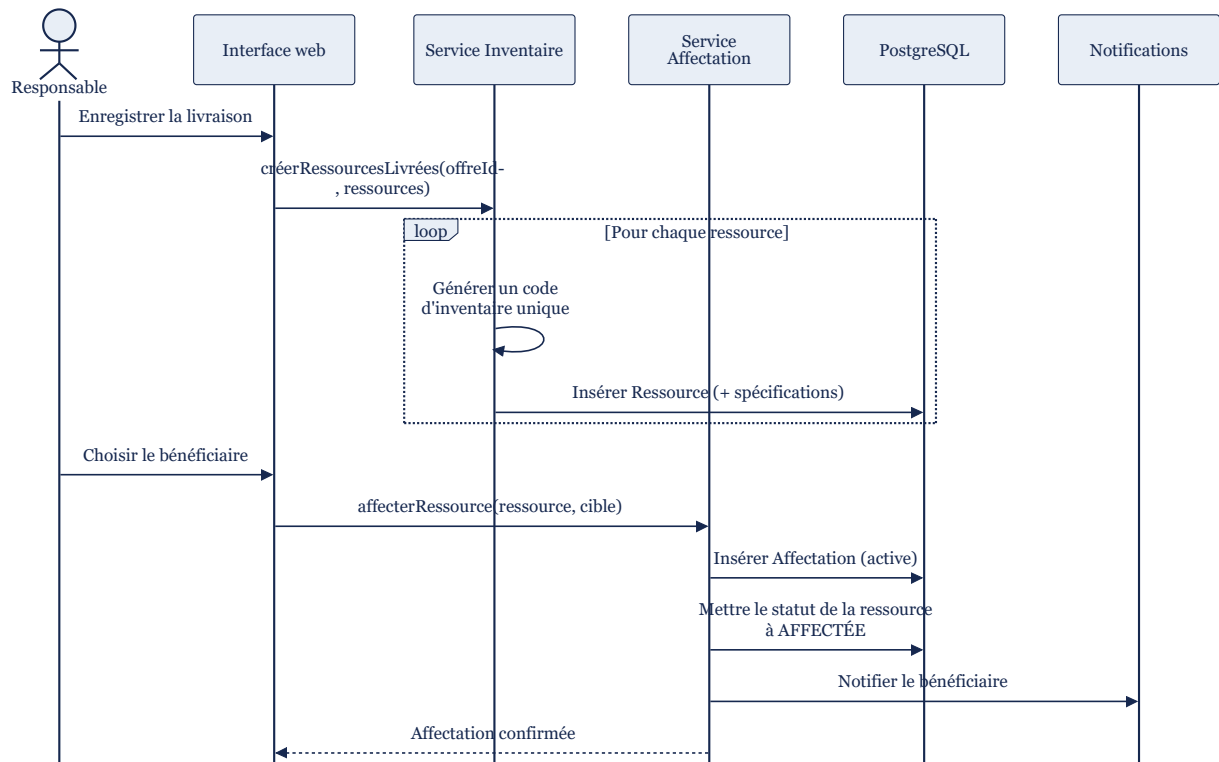


Figure 12.15 — Livraison, génération des codes d'inventaire (BR-06) et affectation.

12.7 Signalement, intervention et constat technique

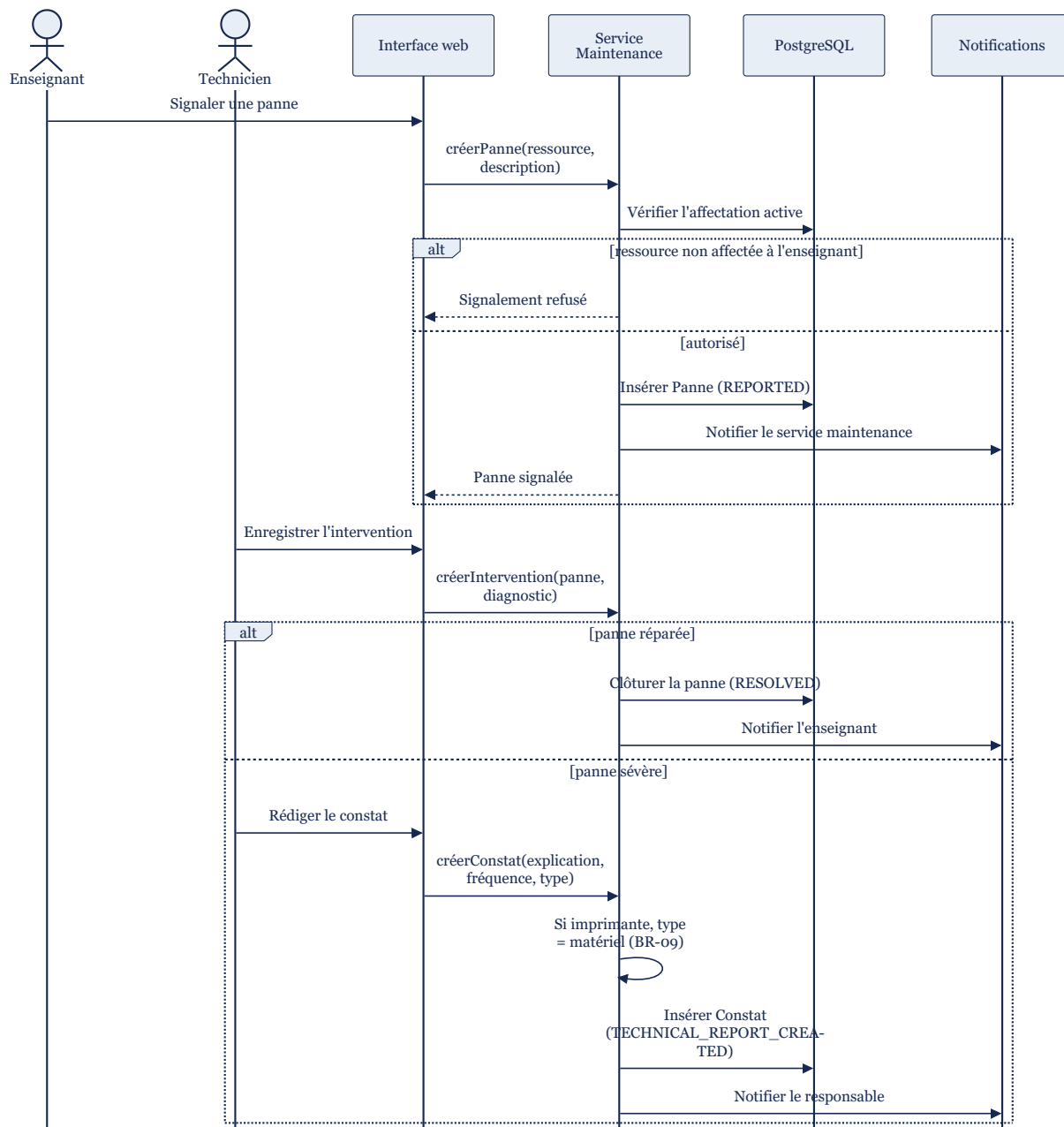


Figure 12.16 — Signalement d'une panne, intervention et rédaction d'un constat (BR-08, BR-09).

12.8 Action de garantie

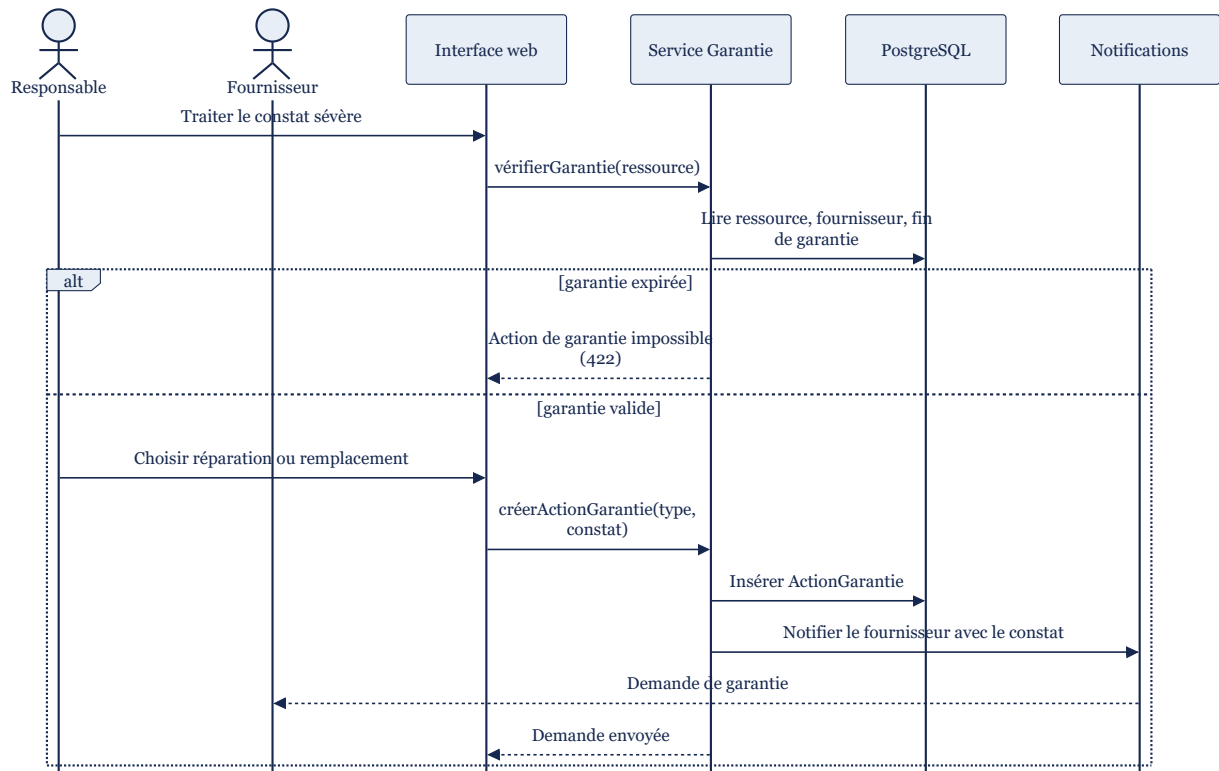


Figure 12.17 — Demande de réparation ou de remplacement sous garantie (BR-10).

13. Tests et démarche qualité

13.1 Stratégie de test en couches

La stratégie de test est **ciblée sur le risque** : l'effort de test se concentre là où réside la complexité sémantique du domaine — les machines à états et l'algorithme de sélection — plutôt que de viser une couverture uniforme et peu significative. Les tests sont organisés en couches.

Tableau 13.15 — *Stratégie de test par couche.*

Couche	Outillage	Cibles
Logique pure / machines à états	<code>bun:test</code>	Transitions de besoin, d'appel d'offre et d'offre ; <code>isTenderActive</code> ; <code>computeOfferTotal</code> ; <code>selectLowestValidOffer</code> ; règle imprimante-matériel ; validité de garantie.
Contrôle d'accès (RBAC)	<code>bun:test</code>	<code>hasPermission</code> pour les combinaisons rôle × permission significatives.
Validation	<code>bun:test</code>	Schémas Valibot (imprimante sans CPU, longueur de mot de passe, dates ISO...).
Primitives d'interface	Vitest + Testing Library	<code>StatusBadge</code> , <code>DataTable</code> (états vide / chargement / données), <code>EmptyState</code> .
Helpers web	Vitest	Permissions, gardes de route, session, gestion d'erreurs, clés de requête, utilitaires de formulaire.
Bout-en-bout (perspective)	Playwright	Parcours « fournisseur → offre », « responsable → acceptation », « enseignant → constat → garantie » (extension future).

13.2 Tests des services purs

Parce que les transitions d'état et la sélection sont isolées dans des fonctions pures, elles se testent sans base de données ni serveur HTTP. C'est ce qui confère la confiance : chaque affirmation du rapport (« l'appel d'offre suit son cycle de vie », « le moins-disant valide gagne », « une panne d'imprimante est nécessairement matérielle ») est **épinglée par un test**. Le projet compte au total quinze fichiers de tests — six côté API (services de besoins, d'appels, d'offres, de maintenance ; matrice de permissions ; schémas) et neuf côté web (tableau de données, badge de statut, état vide, gestion d'erreurs, utilitaires de formulaire, permissions, clés de requête, gardes de route, session).

13.3 Outillage qualité et intégration continue

Au-delà des tests, plusieurs garde-fous assurent la qualité : la **vérification de types** TypeScript de bout en bout (`bun run typecheck`), l'analyse statique et le formatage via `oxlint` / `oxfmt` (outillage Rust très rapide) côté web, et une **chaîne d'intégration continue** (GitHub Actions) qui installe les dépendances, lance la vérification de types, les tests et la construction. La base de données nécessaire à d'éventuels tests d'intégration est provisionnée par le workflow. Les portes de qualité (`bun install` , `typecheck` , `test` , `build`) sont exécutables d'une seule commande à la racine.

14. Déploiement et exploitation

14.1 Conteneurisation

L'ensemble du système est conteneurisé et orchestré par **Docker Compose**. Trois services coopèrent : une base **PostgreSQL 18** (avec volume persistant et sonde de santé `pg_isready`), l'**API** (construite depuis son Dockerfile, sonde de santé sur `/health`), et le **web** (application construite puis servie par nginx). Les dépendances de démarrage sont déclarées : l'API attend que PostgreSQL soit sain, et le web attend que l'API soit saine.

14.2 Diagramme de déploiement

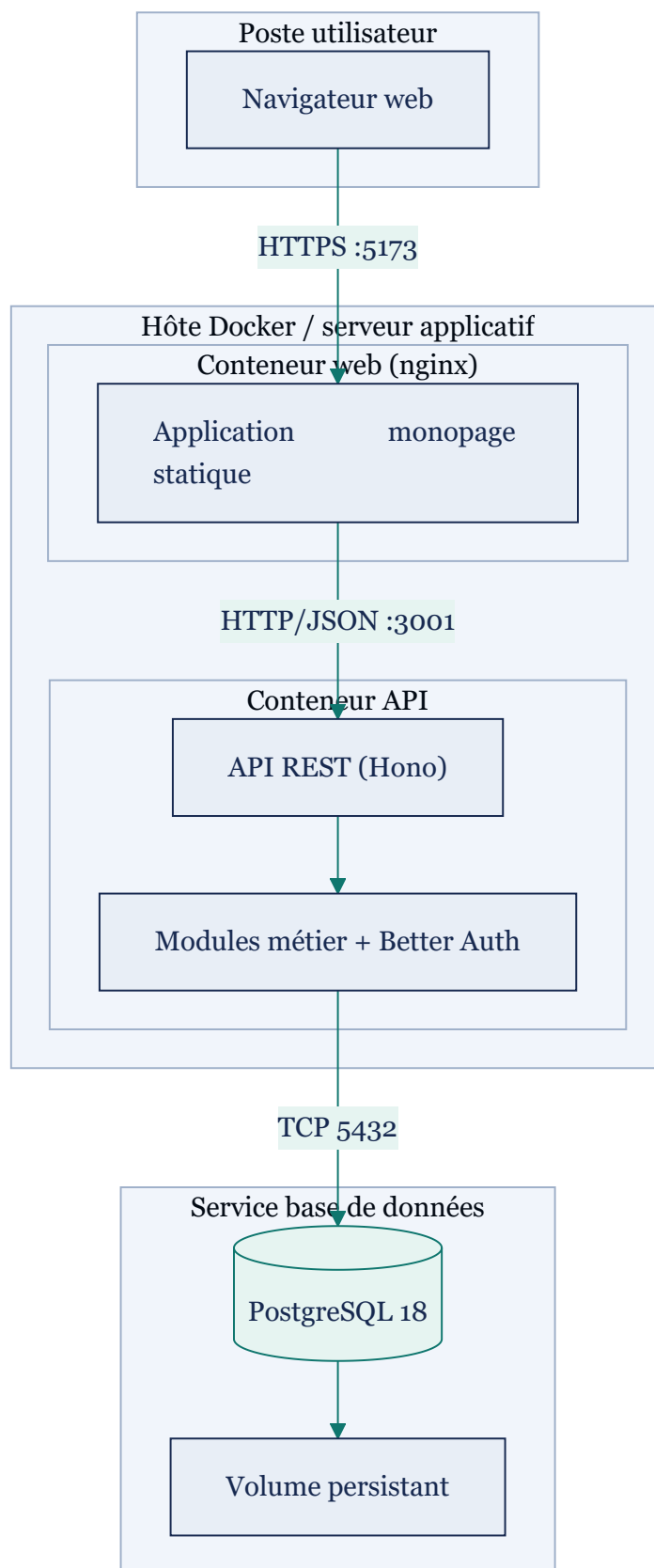


Figure 14.18 — Diagramme de déploiement : navigateur, conteneurs web et API, base PostgreSQL persistante.

14.3 Variables d'environnement

Tableau 14.16 — Variables d'environnement principales.

Variable	Rôle
<code>DATABASE_URL</code>	Chaîne de connexion PostgreSQL.
<code>BETTER_AUTH_SECRET</code>	Secret (≥ 32 caractères) de signature des sessions.
<code>BETTER_AUTH_URL</code>	URL publique de l'API.
<code>TRUSTED_ORIGINS</code>	Origines web autorisées (CORS + cookies).
<code>GITHUB_CLIENT_ID</code> / <code>SECRET</code>	Optionnel — active l'OAuth GitHub.
<code>GOOGLE_CLIENT_ID</code> / <code>SECRET</code>	Optionnel — active l'OAuth Google.
<code>VITE_API_BASE_URL</code>	URL de base de l'API utilisée par le client (au build).

14.4 Mise en route

En développement, la base est lancée en conteneur, les migrations sont appliquées puis le jeu de données initial est chargé, et les deux serveurs (API sur le port 3001, web sur 5173) démarrent en parallèle. La pile complète peut aussi être lancée d'une seule commande.

```
# Développement
bun install
docker compose -f infra/docker-compose.yml up -d postgres
bun run db:migrate      # applique les migrations
bun run db:seed         # charge les données de démonstration
bun run dev             # API :3001 + web :5173

# Pile complète conteneurisée
docker compose -f infra/docker-compose.yml up -d --build # postgres + api + web
```

Extrait 14.3 — Commandes de mise en route en développement et en pile conteneurisée complète.

LISTE DE CONTRÔLE DE DURCISSEMENT (PRODUCTION)

Activer `NODE_ENV=production` (cookies `secure`) ; exécuter derrière un proxy terminant TLS ;
restreindre `TRUSTED_ORIGINS` au domaine réel ; faire tourner régulièrement la sauvegarde PostgreSQL ;
ne jamais committer le secret de session ; externaliser le journal d'audit vers un stockage longue durée.

15. Difficultés rencontrées et solutions

La réalisation a soulevé plusieurs difficultés instructives, principalement à la frontière entre l'authentification, la gestion d'état côté client et la configuration multi-origine. Les exposer honnêtement éclaire des subtilités réelles du développement full-stack moderne.

15.1 La redirection post-connexion (étude de cas)

Symptôme. Après une connexion réussie, l'utilisateur était renvoyé vers `/login` au lieu d'atteindre `/dashboard` ; pourtant, un simple rafraîchissement de la page affichait correctement le tableau de bord. Le bogue était donc strictement *côté client* et lié au *timing*.

Cause racine. La session est stockée dans le cache de TanStack Query avec une fraîcheur (`staleTime`) de trente secondes. Après `signIn.email`, le code appelait `fetchQuery` pour relire la session ; or `fetchQuery` est *sensible au cache* : tant que l'entrée est « fraîche », il renvoie la valeur en cache **sans interroger le réseau**. Comme l'amorçage de l'application avait peuplé le cache avec une session `{ user: null }` encore fraîche, la relecture renvoyait cette valeur périmée. La navigation déclenchait alors le `beforeLoad` de la racine, qui relisait la même session nulle, et la garde `_authenticated` redirigeait vers `/login`. Au rafraîchissement, un cache vierge forçait un véritable appel réseau — avec le cookie désormais présent — d'où le succès.

Solution. Forcer une relecture réseau inconditionnelle sur le seul chemin post-connexion, en surchargeant la fraîcheur à zéro, tout en préservant les trente secondes partout ailleurs :

```
// session.ts – relit et résout la session *fraîche* après connexion
export async function refreshSession(qc) {
  return qc.fetchQuery({ ...sessionQueryOptions, staleTime: 0 })
}
```

Extrait 15.4 — Correctif chirurgical : `staleTime: 0` garantit un appel réseau frais juste après l'authentification.

Leçon. Stocker l'état d'authentification dans un cache applicatif est élégant, mais impose de raisonner explicitement sur la sémantique de fraîcheur : un cache « frais mais périmé sémantiquement » est un piège classique. Le correctif est minimal et localisé, ce qui illustre la valeur d'un diagnostic précis avant toute modification.

15.2 Cookies de session en développement local (HTTP)

Difficulté. Les cookies marqués `secure` ne sont transmis qu'en HTTPS. En développement local (HTTP), un cookie `secure` n'aurait jamais été renvoyé, empêchant toute persistance de session.

Solution. L'attribut `secure` est calculé dynamiquement selon le schéma de l'URL : actif en HTTPS (production), inactif en HTTP (développement). La session persiste ainsi localement sans

compromettre la sécurité en production.

15.3 Configuration multi-origine

Difficulté. En développement, le web (port 5173) et l'API (port 3001) sont des origines distinctes ; par défaut, le navigateur n'attache pas les cookies aux requêtes inter-origines.

Solution. Le serveur autorise explicitement l'origine du client (`TRUSTED_ORIGINS`) et les requêtes authentifiées (`credentials: true`) ; le client émet chaque requête avec `credentials: include`. En production conteneurisée, les deux services partagent le domaine `localhost`, ce qui simplifie le partage du cookie de session.

15.4 Cohérence des autorisations entre couches

Difficulté. Le risque, dans toute application full-stack, est que les règles d'autorisation du client et du serveur divergent au fil des évolutions.

Solution. La matrice de permissions est définie une seule fois dans le package partagé et importée des deux côtés. Toute évolution s'applique simultanément au serveur et au client, éliminant par construction la classe de bogues de désynchronisation.

16. Bilan, limites et perspectives

16.1 Bilan

Le système réalisé couvre l'intégralité du périmètre spécifié et satisfait les exigences structurantes. Les six processus métier sont opérationnels de bout en bout ; les règles de gestion sont implémentées et, pour les plus critiques, vérifiées par des tests ; les exigences non fonctionnelles de sécurité, de contrôle d'accès, de traçabilité et d'intégrité sont prises en charge par construction. L'architecture en monolithe modulaire s'est révélée adaptée : elle a permis une progression rapide et une bonne maintenabilité, sans coût opérationnel superflu.

Tableau 16.17 — Couverture des exigences non fonctionnelles par la réalisation.

Exigence	Réponse technique	État
Sécurité (NFR-01)	Better Auth, sessions par cookie, mots de passe hachés, aucun secret en clair.	Couverte
Contrôle d'accès (NFR-02)	Matrice RBAC partagée, vérifiée côté serveur.	Couverte
Traçabilité (NFR-03)	Journal d'audit avec valeurs avant/après.	Couverte
Intégrité (NFR-04)	Clés étrangères, unicité, machines à états, transactions.	Couverte
Maintenabilité (NFR-08)	Monolithe modulaire, package partagé, tests des services purs.	Couverte
Ergonomie (NFR-09)	Tableau de bord par rôle, libellés humains, états de chargement.	Couverte
Confidentialité (NFR-10)	Portées de permission ; un fournisseur ne voit que ses offres.	Couverte

16.2 Limites

- Le **contrôle budgétaire** par département n'est pas implémenté (hors périmètre assumé).
- Les **notifications** sont uniquement internes ; aucun canal courriel ou SMS.
- La couverture **bout-en-bout** (Playwright) reste à constituer ; les tests actuels portent sur la logique pure et les composants.
- Le **périmètre matériel** se limite aux ordinateurs et aux imprimantes.
- Les seuils de **performance** et de **disponibilité** sont calibrés pour un prototype académique.

16.3 Perspectives

- **Notifications externes** : ajouter un canal courriel (et éventuellement des préférences par utilisateur).
- **Contrôle budgétaire** : associer un budget aux départements et bloquer les besoins qui le dépassent.
- **Tests bout-en-bout** : couvrir les trois parcours « en or » (fournisseur → offre, responsable → acceptation, enseignant → constat → garantie).
- **Observabilité** : métriques, traçage distribué et externalisation du journal d'audit vers un stockage longue durée.
- **Extensibilité** : la matrice de permissions étant une donnée et chaque module suivant un même squelette, l'ajout de nouveaux types de ressources ou de nouveaux modules suit un chemin balisé.

17. Conclusion

Ce rapport technique a décrit la conception et la réalisation d'un système full-stack de gestion des ressources matérielles d'une faculté, depuis les choix d'architecture jusqu'au déploiement conteneurisé. La solution adopte un monolithe modulaire trois tiers, un choix justifié par le périmètre et porteur des qualités attendues : séparation claire des responsabilités, maintenabilité, testabilité et intégrité.

Les décisions techniques ont été constamment reliées aux exigences qu'elles servent : la matrice de permissions partagée garantit le contrôle d'accès et la confidentialité ; les machines à états et l'algorithme de sélection, isolés dans des services purs et testés, garantissent l'intégrité métier ; le journal d'audit assure la traçabilité ; et la conteneurisation rend le système reproductible et déployable. Les difficultés rencontrées, notamment autour de la gestion d'état d'authentification côté client, ont été diagnostiquées avec rigueur et résolues par des correctifs minimaux et ciblés.

Au regard du cahier des charges, l'ensemble des processus métier et des règles de gestion a été couvert et, pour les plus critiques, vérifié. Le projet démontre ainsi une **maîtrise de bout en bout** de la chaîne d'ingénierie : de l'analyse du besoin à une application fonctionnelle, sécurisée, testée et déployable. Les perspectives identifiées (notifications externes, contrôle budgétaire, tests bout-en-bout, observabilité) tracent une trajectoire d'évolution naturelle vers un usage en production.

18. Annexes

Annexe A — Comptes de démonstration

Le jeu de données initial provisionne un compte par rôle (et plusieurs variantes) afin que chaque tableau de bord dispose de contenu. Tous les comptes partagent le mot de passe `changeme123` et n'existent qu'en environnement local.

Tableau 18.18 — Comptes de démonstration (extrait).

Email	Rôle	Département / Société
<code>admin@faculty.local</code>	ADMIN	—
<code>manager@faculty.local</code>	RESOURCE_MANAGER	—
<code>cs.head@faculty.local</code>	DEPARTMENT_HEAD	Informatique (CS)
<code>teacher@faculty.local</code>	TEACHER	Informatique (CS)
<code>tech@faculty.local</code>	MAINTENANCE_TECHNICIAN	—
<code>supplier@faculty.local</code>	SUPPLIER	Acme Hardware

Annexe B — Commandes utiles

```
# Portes de qualité
bun install
bun run typecheck      # vérification de types (API + web)
bun run test           # tests (API + web)
bun run build          # construction des deux artefacts

# Base de données
bun run db:generate    # génère une migration depuis le schéma
bun run db:migrate     # applique les migrations
bun run db:seed        # charge les données de démonstration

# Pile Docker
docker compose -f infra/docker-compose.yml up -d --build
docker compose -f infra/docker-compose.yml ps
```

Annexe C — Références

- Documentation Hono — framework HTTP TypeScript.
- Documentation Better Auth — authentification et sessions.
- Documentation Drizzle ORM — schéma, requêtes et migrations.
- Documentation TanStack (Router, Query, Form) — gestion du routage et de l'état côté React.

- Documentation PostgreSQL — système de gestion de base de données.
- Cahier des charges et dossier de spécifications du projet (document associé).
- Code source du projet — github.com/Mohamed-1954/material-resources-management